



Preface

- ❖ This manual is an introduction to overview of the Keil development tools Knowledge about 8051 microcontrollers.
- ❖ This is works Integrated Development Environment Shortly called as IDE.
- ❖ This manual helps step by step procedure of programming, debugging, with software and implementation with hardware.

Chapter Overview

Chapter 1	: 8051 Introduction
Chapter 2	: Micro controller Architecture
Chapter 3	: Development Board & Interfacing
Chapter 4	: Creating 8051 Programs
Chapter 5	: Debugging
Chapter 6	: Using Target Hardware
Chapter 7	: Example Programs
Chapter 8	: Software implementation and Explanation
Chapter 9	: Hardware Implementation and Explanation
Chapter 10	: Work Exercise

Chapter 1

8051 Introduction

This book, Getting Started, depicts the μ Vision IDE, μ Vision Debugger and Investigation Tools, the recreation, and troubleshooting and following abilities. In expansion to depicting the fundamental conduct and essential screens of μ Vision, this book gives an exhaustive diagram of the upheld microcontroller engineering types, their points of interest and features, and supports you in choosing the fitting objective gadget. This book joins clues to assist you with composing better code. Likewise with any Getting Started book, it doesn't cover each perspective and the numerous accessible setup choices in detail. We urge you to work through the guides to get acquainted with μ Vision and the segments conveyed. The Keil Development Tools are intended for the expert programming engineer, anyway developers, everything being equal, can utilize them to benefit from the implanted microcontroller designs that are upheld. Devices created by Keil embrace the most well known microcontrollers and are disseminated in a few bundles and designs, subject to the engineering.

Chapter 2

Microcontroller Architectures

Introduction to 8051 Architectures

The 8051 Architecture consist of these features

- ✓ The 8 bit CPU with Registers A and B
- ✓ Internal ROM.
- ✓ 16-bit program counter(PC) and data pointer(DPTR).
- ✓ Internal RAM of 128 bytes.
- ✓ 8-bit Program Status word(PSW).
- ✓ Two 16 bit Counter / timers.
- ✓ 4 eight-bit ports.
- ✓ 3 internal interrupts and 2 external interrupts.
- ✓ Control register.
- ✓ Oscillator and clock circuits.
- ✓ A and B CPU Register
- ✓ The 8051 contains 34 general purpose or working registers. Two of these Register A and B.
- ✓ The immediate result is stored in the accumulator register (Acc) for next operation
- ✓ The B register is a register just for multiplication and division operation which requires more register spaces for the product of multiplication and the quotient and the remainder for the division.
- ✓ Program status word(PSW)
- ✓ The PSW contain the math flags, User program flagF0,and the register select bits that identify which of the four General-purpose register banks is currently in use by the program
- ✓ The math flags include carry(c),auxiliary carry(AC),overflow(OV) and parity(p)
- ✓ **Program Status Word (PSW)**

- ✓ The 8051 oscillator and clock
- ✓ The 8051 requires an external oscillator circuit. The oscillator circuit usually runs around 12MHz. The crystal generates 12M pulses in one second.
- ✓ A machine cycle is minimum amount time must take by simplest machine instruction
- ✓ An 8051 machine cycle consists of 12 crystal pulses (clock cycle)
- ✓ The first 6 crystal pulses (clock cycle) is used to fetch the Opcode and the second 6 pulses are used to perform the operation on the operands in the ALU.
- ✓ This gives an effective machine cycle rate at 1MIPS (Million Instructions Per Second).
- ✓ Oscillator circuit and timing
- ✓ **Program counter PC** As the CPU fetches the opcode from the program ROM, the program counter is increasing to point to the next instruction. The program counter points to the address of the next instruction to be Executed. As the CPU fetches the opcode from the program ROM, the program counter is increasing to point to the next instruction
- ✓ **The program Counter** is 16 bits wide means that it can access program addresses 0000 to FFFFH, a total of 64K bytes of code
- ✓ **Data pointer (DPTR)** The data pointer is 16 bit register It is used to hold the address of the data in the memory The DPTR register can be accessed separately as lower eight bit(DPL) and higher eight bit (DPH) It can be used as a 16 bit data register or two independent data register.
- ✓ **Stack pointer (SP)** The stack is a section of RAM used by the CPU to store information temporarily %The register used to access the stack is called the SP(stack pointer) register—This information could be data or an address The stack pointer in the 8051 is only 8 bit wide.
- ✓ Operation of stack pop push stack pointer stack

- ✓ Internal memory 128 bytes of RAM Directly addressable range: 00 to 7F hexadecimal. Indirectly addressable range: 00 to FF hexadecimal Bit addressable space: 20 to 2F hexadecimal Four register banks: 00 to 1F hexadecimal.
- ✓ Internal memory - Internal RAM 128 byte ,Thirty-two bytes from address 00h to 1Fh that make up 32 working register organized as Four bank of eight bit each. Bits RS0 and RS1 in the PSW determine which bank of register is currently in use. Bank 0 is selected upon reset².Bit addressable: A bit addressable area of 16 byte so occupies RAM bytes addresses 20h to 2Fh,forming A total of 128 addressable bits. An addressable bit may be specified by its bit address of 00h to 7Fh.³.General purpose: A general-purpose RAM area above the bit area, form 30h to 7Fh ,addressable as bytes.
- ✓ **Internal RAM Organization**
- ✓ **External memory** External memory is used in cases when the internal ROM and RAM memory Available On chip is not sufficient. Two separate are made available by the 16-bit PC and the DPTR and by different control pins for enabling external ROM and RAM chips. If the 128 bytes of internal RAM is insufficient, the external RAM is accessed by the DPTR. In the 8051 family, external RAM of up to 64 KB can be added to any chip.
- ✓ **Special Function Register (SFR)** The SFR (Special Function Register) can be accessed by their names or by their addresses. The SFR registers have addresses between 80H and FFH. Not all the address space of 80 to FF is used by SFR. The unused

- ✓ locations 80H to FFH are reserved and must not be used by the 8051 programmer
- ✓ **Special Function Register** Map Bit Addressable F8 F0 B E8 E0 ACC D8 D0 PSW C8 C0 B8 IP B0P3 A8 IE A0 P2 98 SCON SBUF 90 P1 88 TCON TMOD TL0 TL1 TH0 TH1 80 P0 SP DPL DPH PCON Special Function Register (SFR)cont.. Special Function Register (SFR)cont.. 128 byte address space, directly addressable as 80 to FF hex. 16 addresses are bit addressable.
- ✓ **Special purpose CPU registers.** I/O ports. Interrupt control Timers serial I/O Special Function Register (SFR) CPU registers:
 - ACC : Accumulator. -B : B register. -PSW : Program Status Word. - SP : Stack Pointer. - DPTR : Data Pointer (DPH, DPL).
 - Interrupt control: -IE : Interrupt Enable. -IP : Interrupt Priority.
 - I/O Ports: - P0 : Port 0. - P1 : Port 1. - P2 :Port 2. - P3 : Port 3.
- ✓ **Special Function Register (SFR)** Timers: - TMOD : Timer mode. - TCON : Timer control. -TH0 : Timer 0 high byte. - TL0 : Timer 0 low byte. - TH1 : Timer 1 high byte. - TL1 : Timer 1 low byte. Serial I/O: - SCON : Serial port control. - SBUF : Serial data registers. Other: - PCON :Power control
- ✓ I/O Ports-Four 8-bit I/O ports. Port 0 Port 1 Port 2 Port 3- Most have alternate functions.- Quasi-bidirectional
- ✓ Port 0- Port 0 is a dual purpose port, it is located from pin 32 to pin 39 (8 pins).- To use this port as both input/output ports each pin must be connected externally to pull-up resistor.- As an I/O port.-Alternate functions: As a multiplexed data bus. 8-bit instruction bus, strobed by PSEN. Low byte of address bus,

- ✓ strobed by ALE. 8-bit data bus, strobed by WR and RD.
- ✓ **Port 1-** Port 1 is a dedicated I/O port from pin 1 to pin 8.- Upon reset it is configured as out port.- It is generally used for interfacing to external device- thus if you need to connect to switches or LEDs, you could make use of these 8 pins,- but it doesn't need any pull-up resistors as it is having internally-As an I/O port: Standard quasi-bidirectional.
- ✓ **Port 2-** Like port 0, port 2 is a dual-purpose port.(Pins 21 through 28)- It can be used for general I/O or as the high byte of the address bus for designs with external code memory.- Like P1 ,Port2 also doesn't require any pull-up resistors- As an I/O port: Standard quasi-bidirectional.- Alternate functions: High byte of address bus for external program and data memory accesses.
- ✓ **Port 3-** **Port 3** is also dual purpose but designers generally avoid using this port unnecessarily for I/O because the pins have alternate functions which are related to special features of the 8051.-Indiscriminate use of these pins may interfere with the normal operation of the 8051.- As an I/O port: Standard quasi-bidirectional.- Alternate functions: Serial I/O - TXD, RXD Timer clocks - T0, T1Interrupts - INT0, INT1 Data memory- RD, WR
- ✓ **I/O Port Structure** The internal circuitry for the I/O port is shown in the figure If you want to read in from a pin, you must first give a logic '1' to the port latch to turn off the FET otherwise the data read in will always be logic '0'. When you write to the port you are actually writing to the latch e.g. a logic 0 given to the latch will be inverted and turn on the FET which cause the port pin to be connected to Gnd (logic 0).

✓ **Timer/Counters** Two 16-bit up counters, named T0 and T1, are provided for the general use of the programmer. Each counter may be programmed to count internal clock pulses, acting as a timer, or programmed to count external pulses as a counter. The counters are divided into two 8-bit registers called the timer low (TL0, TL1) and high (TH0, TH1) bytes. All counter action is controlled by bit states in the timer mode control register (TMOD), the timer/counter control register (TCON) and certain program instructions. TMOD is dedicated to the two timers and can be considered two duplicate 4-bit registers, each of which controls the action of the timers. TCON has control bits and flags for the timers in the upper control bits and flags for the external interrupts in the lower nibble. These timers exist in the SFR area as pairs of 8-bit registers.— TL0 (8AH) and TH0 (8CH) for Timer0.— TL1 (8BH) and TH1 (8DH) for Timer1. (LSB is bit 0 ; MSB is bit 7) When used as timers, the registers are incremented once per machine cycle. — Each machine cycle is 12 clock cycles. Count frequency = (system clock frequency) / 12 When used as counters, the registers will be incremented once on every 1-0 (negative edge) on the appropriate input pin. • T0 — P3.4 • T1 — P3.5 The pins must be held high for one complete machine cycle and then low for one complete machine cycle.

✓ **Timer/Counters: Application** The timers can be used for: 1. Interval timing The timer is programmed to overflow at a regular interval and set the timer overflow flag. Overflow means reaching maximum count of FFFFH. 2. Event counting Determine the number of occurrences of an event. An event is any external stimulus that provides a 1-to-0 transition on a pin of the C.

✓ **TCON** (Counter/Timer Control Register) TF1 TR1 TF0 TR0 IE1 IT1 IE0 IT0 - TF1, TF0 : Overflow flags for Timer 1 and

- ✓ Timer 0. - TR1, TR0 : Run control bits for Timer 1 and Timer 0. Set to run, reset to hold. - IE1, IE0 : Edge flag for external interrupts 1 and 0. Set by interrupt edge, cleared when interrupt is processed. - IT1, IT0 : Type bit for external interrupts. * Set for falling edge interrupts, reset for 0 level interrupts. * = not related to counter/timer operation.
- ✓ **TCON** (Counter/Timer Control Register) TF1 TR1 TF0 TR0 IE1 IT1 IE0 IT0 - TF1, TF0 :Overflow flags for Timer 1 and Timer 0. - TR1, TR0 : Run control bits for Timer 1 and Timer 0. Set to run, reset to hold. - IE1, IE0 : Edge flag for external interrupts 1 and 0. Set by interrupt edge, cleared when interrupt is processed. - IT1, IT0 : Type bit for external interrupts. * Set for falling edge interrupts, reset for 0 level interrupts. * = not related to counter/timer operation.
- ✓ **TMODGATE** C/T M1 M0 GATE C/T M1 M0 Timer 1 Timer 0 - GATE : Permits INT x pin to enable/disable counter. - C/T : Set for counter operation, reset for timer operation. M1, M0 : Operating Mode select Bit 1/0. Set/Cleared by program to select Mode M1 M0 Mode 0 0 0 0 1 1 1 0 2 1 1 3
- ✓ Interrupt System 5 Interrupt Sources (in order of priority): 1 External Interrupt 0 (IE0) 2 Timer 0(TF0) 3 External Interrupt 1 (IE1) 4 Timer 1 (TF1) 5 Serial Port (RI/TI) Each interrupt type has a separate vector address. Each interrupt type can be programmed to one of two priority levels. External interrupts can be programmed for edge or level sensitivity.
- ✓ Interrupt vector Addresses Source Address IE0 03H TF0 0BH IE1 13H TF1 1BH RI&TI 23H The8051 starts execution at 0000H after Reset.

✓

✓ IE : Interrupt Enable Register EA ---- ES ET1 EX1 ET0 EX0
- EA : Global interrupt enable. -ES : Enable serial port interrupt -
ET1 : Timer 1. - EX1 : External interrupt 1. - ET0 : Timer 0. -
EX0 : External interrupt 0. - 0 = Disabled. - 1 = Enabled.

✓ IP: Interrupt Priority Register ----- PS PT1 PX1 PT0
PX0 - PS : Serial interface. - PT1 : Timer 1. - PX1 : External
interrupt 1. - PT0 : Timer 0. - PX0 : External interrupt 0. - 0 =
Low priority. -1 = High priority.

✓ Types of Serial communications

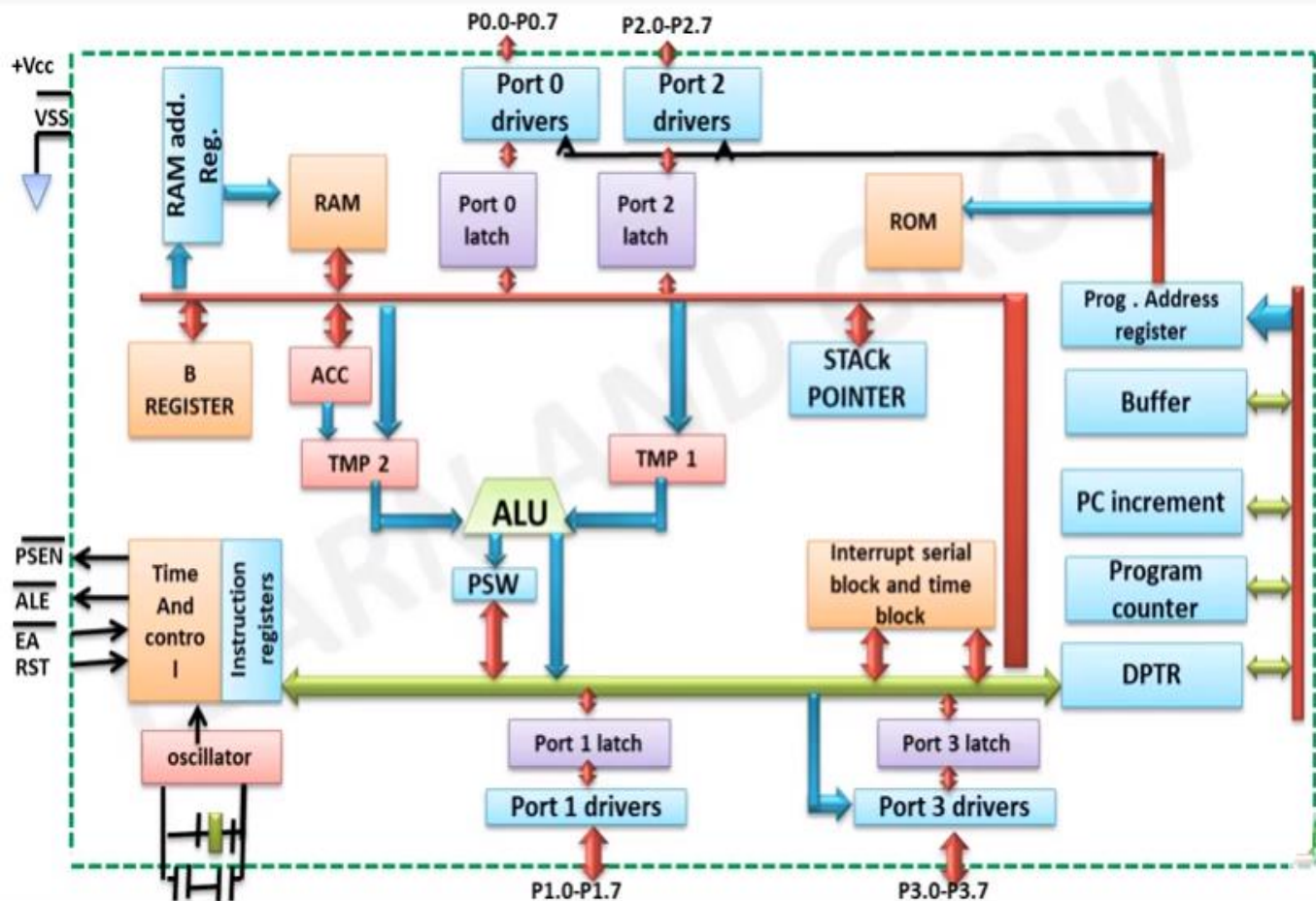
✓ Rx/D and Tx/D pins in the 8051. The 8051 has two pins for transferring and receiving data by serial communication. These two pins are part of the Port3 (P3.0 & P3.1). These pins are TTL compatible and hence they require a line driver to make them RS232 compatible. Serial communication is controlled by an 8-bit register called SCON register, it is a bit addressable register.

✓ SCON : Serial Control Register SMO SM1 SM2 REN TB8 RB8
TI RI - SM0, SM1 = Serial Mode: 00 = Mode 0 : Shift register I/O
expansion. 01 = Mode 1 : 8-bit UART with variable baud rate. 10
= Mode 2 : 9-bit UART with fixed baud rate. 11 = Mode 3 : 9-bit
UART with variable baud rate. - SM2 : It enables the
multiprocessor communication feature in Mode 2 & Mode 3 -
REN = Enables receiver. - TB8 = Ninth bit transmitted (in modes
2 and 3). - RB8 = Ninth bit received: Mode 0 : Not used. Mode 1 :
Stop bit. Mode 2, 3 : Ninth data bit. - TI = Transmit interrupt flag.
- RI = Receive interrupt flag.

- ✓ SM0 , SM1 These two bits of SCON register determine the framing of data by specifying the number of bits per character and start bit and stop bits. There are 4 serial modes. SM0 SM1 Serial Mode 0 Serial Mode 1, 8 bit data, 1 stop bit, 1 start bit 1 0 Serial Mode 2 Serial Mode 3
- ✓ REN• REN (Receive Enable) also referred as SCON.4. When it is high, it allows the 8051 to receive data on the Rx D pin. So to receive and transfer data REN must be set to 1. When REN=0, the receiver is disabled. This is achieved as below SETB SCON.4 & CLR SCON.4
- ✓ TI , RI• TI (Transmit interrupt) is the D1 bit of SCON register. When 8051 finishes the transfer of 8-bit character, it raises the TI flag to indicate that it is ready to transfer another byte. The TI bit is raised at the beginning of the stop bit. RI (Receive interrupt) is the D0 bit of the SCON register. When the 8051 receives data serially, via Rx D, it gets rid of the start and stop bits and places the byte in the SBUF register. Then it raises the RI flag bit to indicate that a byte has been received and should be picked up before it is lost. RI is raised halfway through the stop bit.
- ✓ Serial Interface Full duplex UART. Four modes of operation: 1. Synchronous serial I/O expansion. 2. Asynchronous serial I/O with variable baud rate. 3. Nine bit mode with variable baud rate. 4. Nine bit mode with fixed baud rate. 10 or 11 bit frames. Registers: SCON - Serial port control register. SBUF - Read received data. - Write data to be transmitted. PCON - SMOD bit.
- ✓ Serial Interface Modes of Operation TXD and RXD are the serial output and input pins (Port 3, bits 1 and 0). Mode 0: Shift Register Mode. Serial data is transmitted/received on RXD. TXD outputs

- ✓ shift clock. Baud Rate is $1/12$ of clock frequency. Mode 1:10-bits transmitted or received. Start (0), 8 data bits (LSB first), and a stop bit (1). Baud Rate Clock is variable using Timer 1 overflow or external count input. Can go up to 104.2 KHz (20 MHz).
- ✓ Serial Interface Modes of Operation(cont.) Mode 2:11-bits transmitted or received. Start (0), 8 data bits (LSB first), programmable 9th bit, and stop bit (1). Baud Rate programmable to either $1/32$ or $1/64$ oscillator frequency (625 KHz for 20 MHz osc.). Mode 3:11-bit mode. Baud Rate variable using Timer 1 overflow or external input. 104.2 KHz max. (20 MHz)
- ✓
- ✓ . Multi-Drop Communication Serial Communication Modes 2 and 3 allow one "Master" 8051 to control several "Slaves": The serial port can be programmed to generate an interrupt if the 9th data bit = 1. The TXD outputs of the slaves are tied together and to the RXD input of the master. The RXD inputs of the slaves are tied together and to the TXD output of the master. Each slave is assigned an address. Address bytes transmitted by the master have the 9th bit = 1.
- ✓ The Addressed slave can then carry out the masters commands. When the master transmits an address byte, all the slaves are interrupted. The slaves then check to see if they are being addressed or not.

8051 Architecture Diagram



8051 Architecture Advantages

- Fast I/O operations and fast access to on-chip RAM in data space
- Efficient and flexible interrupt system
- Low-power operation

8051-based devices are typically used in small and medium sized applications that require high I/O throughput. Many devices with flexible peripherals are available, even in the smallest chip packages.

8051 Highlights

- Fast interrupt service routines with two or four priority levels and up to 32- vectored interrupts.
- Four register banks for minimum interrupt prolog/epilog Bit-addressable space for efficient logical operations.
- 128 Bytes of Special Function Register (SFR) space for tight integration of on-chip peripherals. Some devices extend the SFR space using paging.
- Low-power, high-speed devices up to 100 MIPS are available.

8051 Development Tool Support

The Keil C51 Compiler and the Keil Linker/Locator provide optimum 8051 architecture support with the following features and C language extensions.

- Interrupt functions with register bank support are written directly in C

Bit and bit-**addressable** variables for optimal Boolean data type support

- Compile-time stack with data overlaying uses direct memory access and gives high-speed code with little overhead compared to assembly programming
- Reentrant functions for usage by multiple interrupt or task threats Generic and memory-specific pointers provide flexible memory access Linker Code Packing gives utmost code density by reusing identical program sequences
- Code and Variable Banking expand the physical memory address space
- Absolute Variable Locating enables peripheral access and memory sharing

Chapter 3

Development Board & Interfacing Accessories :

DEVELOPMENT BOARD : 8051BOARD (P89V51RD2)

INTERFACING ACCESSORIES : LED, LCD, KEYPAD, ADC, DAC,
ADC, DAC, STEPPER MOTOR.

SENSORS : ZIGBEE, GPS, GSM,
BLUETOOTH, PIR, IR, RFID.



Chapter 4

Instruction sets

DATA TRANSFER	ARITHMETIC	LOGICAL	BOOLEAN	PROGRAM BRANCHING	DATA TRANSFER
MOV	ADD	ANL	CLR	LJMP	MOV
MOVC	ADDC	ORL	SETB	AJMP	MOVC
MOVX	SUBB	XRL	MOV	SJMP	MOVX
PUSH	INC	CLR	JC	JZ	PUSH
POP	DEC	CPL	JNC	JNZ	POP
XCH	MUL	RL	JB	CJNE	XCH
XCHD	DIV	RLC	JNB	DJNZ	XCHD
	DA A	RR	JBC	NOP	
		RRC	ANL	LCALL	
		SWAP	ORL	ACALL	
			CPL	RET	
				RETI	
				JMP	

Alphabetical List of Instructions

ACALL	- Absolute Call	MOV	- Move Memory
ADD, ADDC	- Add Accumulator (With Carry)	MOVC	- Move Code Memory
AJMP	- Absolute Jump	MOVX	- Move Extended memory
ANL	- Bitwise AND	MUL	- Multiply Accumulator by B
CJNE	- Compare and Jump if Not Equal	NOP	- No Operation
CLR	- Clear Register	ORL	- Bitwise OR
CPL	- Complement Register	POP	- Pop Value From Stack
DA	- Decimal Adjust	PUSH	- Push Value Onto Stack
DEC	- Decrement Register	RET	- Return From Subroutine
DIV	- Divide Accumulator by B	RETI	- Return From Interrupt
DJNZ	- Decrement Register and Jump if Not Zero	RL	- Rotate Accumulator Left
INC	- Increment Register	RLC	- Rotate Accumulator Left Through Carry
JB	- Jump if Bit Set	RR	- Rotate Accumulator Right
JBC	- Jump if Bit Set and Clear Bit	RRC	- Rotate Accumulator Right Through Carry
JC	- Jump if Carry Set	SETB	- Set Bit
JMP	- Jump to Address	SJMP	- Short Jump
JNB	- Jump if Bit Not Set	SUBB	- Subtract From Accumulator With Borrow
JNC	- Jump if Carry Not Set	SWAP	- Swap Accumulator Nibbles
JNZ	- Jump if Accumulator Not Zero	XCH	- Exchange Bytes
JZ	- Jump if Accumulator Zero	XCHD	- Exchange Digits
LCALL	- Long Call	XRL	- Bitwise Exclusive OR
LJMP	- Long Jump	Undefined	- Undefined Instruction

Chapter 5

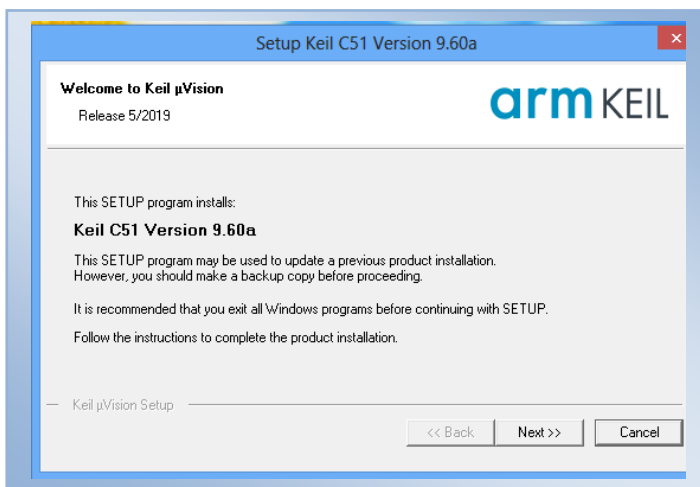
Keil software Tool

<https://www.keil.com/demo/eval/c51.htm> - Register your own Details and get download the product.

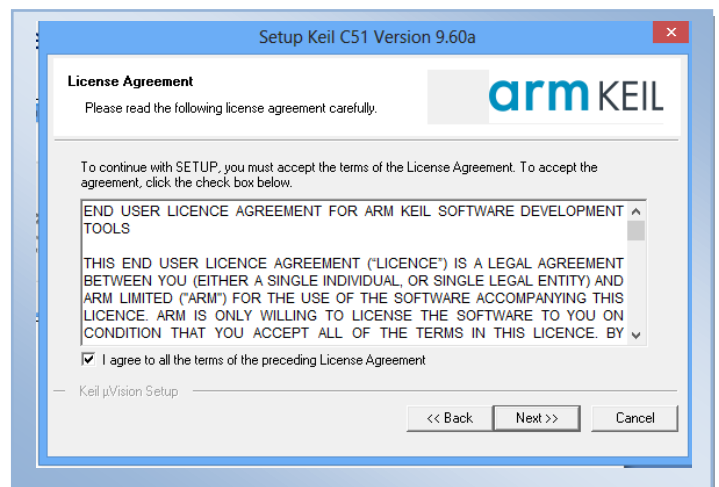
Install the software your machine

Step by step procedure to install keil software

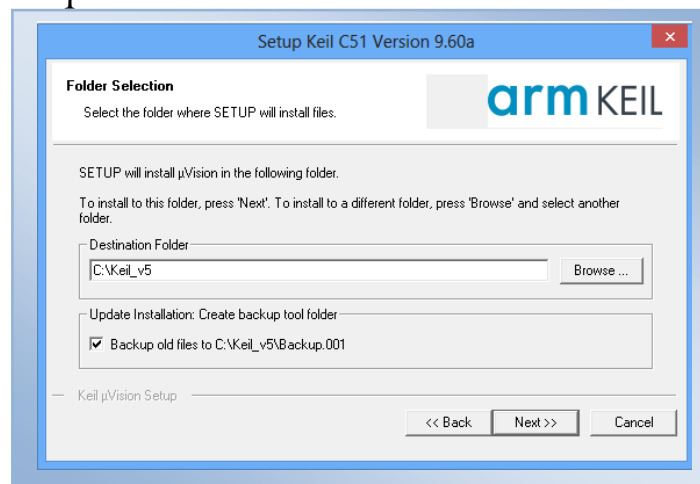
Step 1



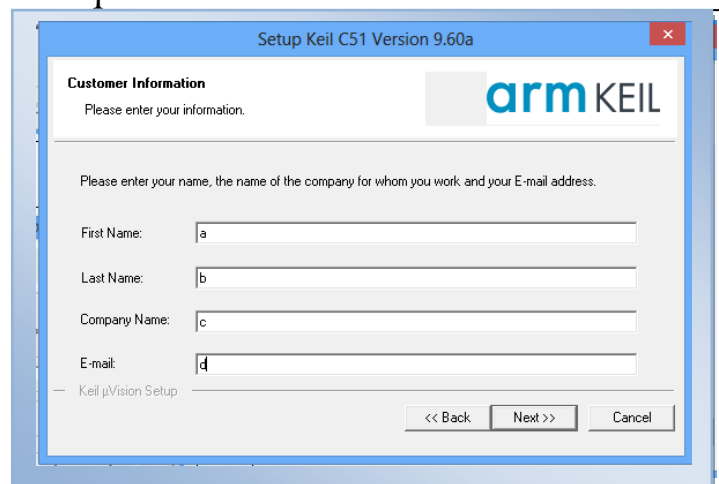
Step 2



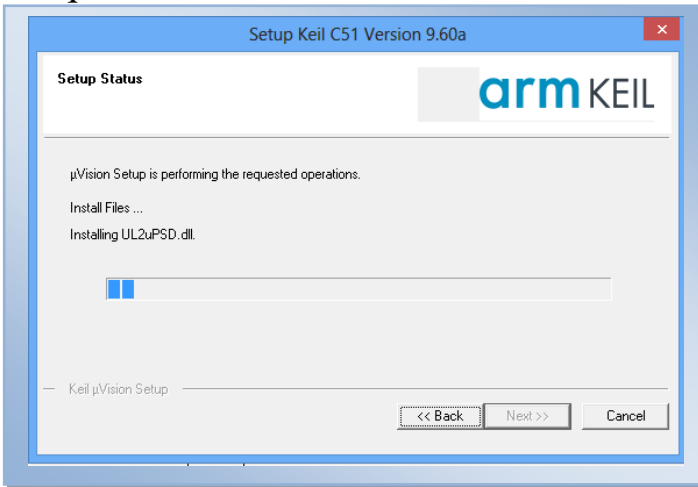
Step 3



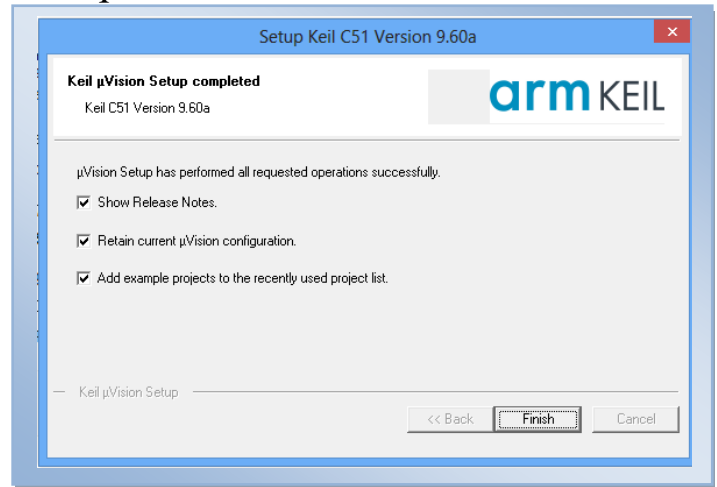
Step 4



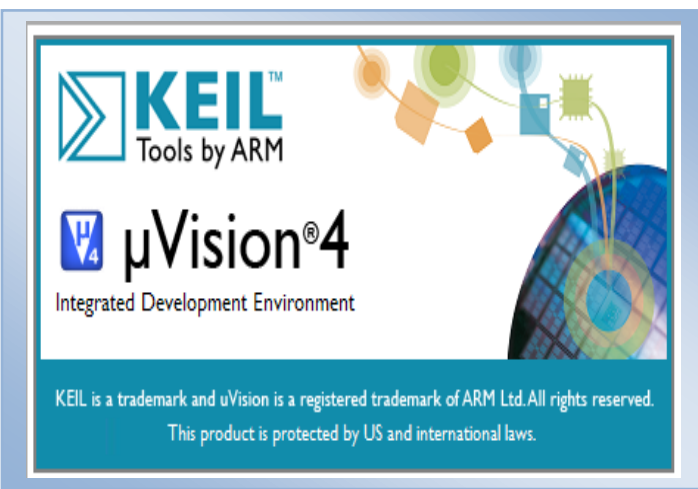
Step 5



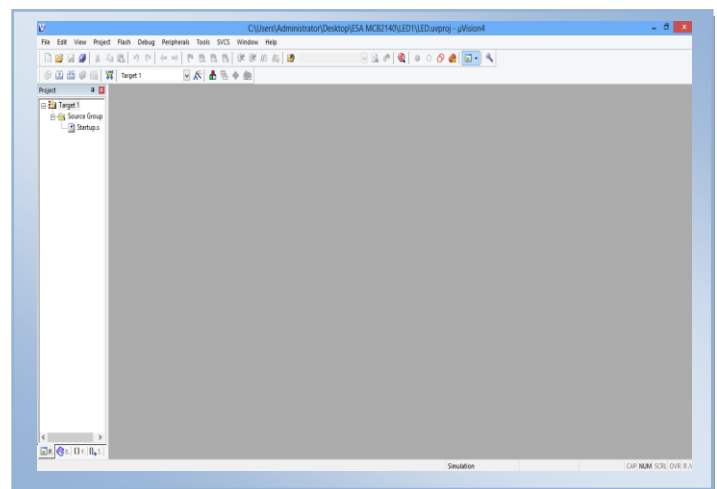
Step 6



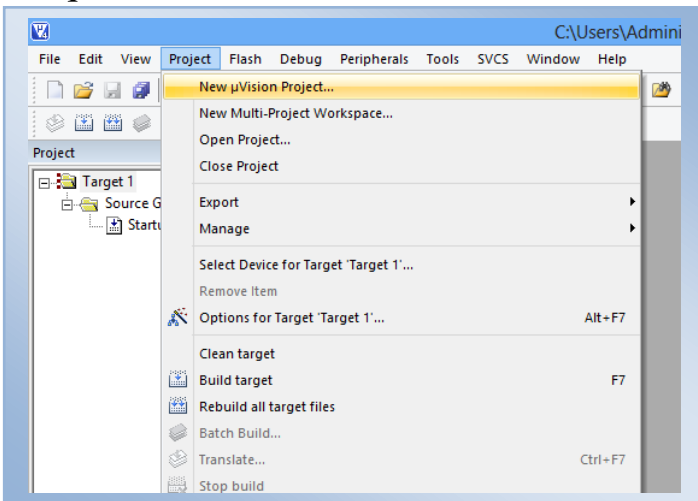
Step 7



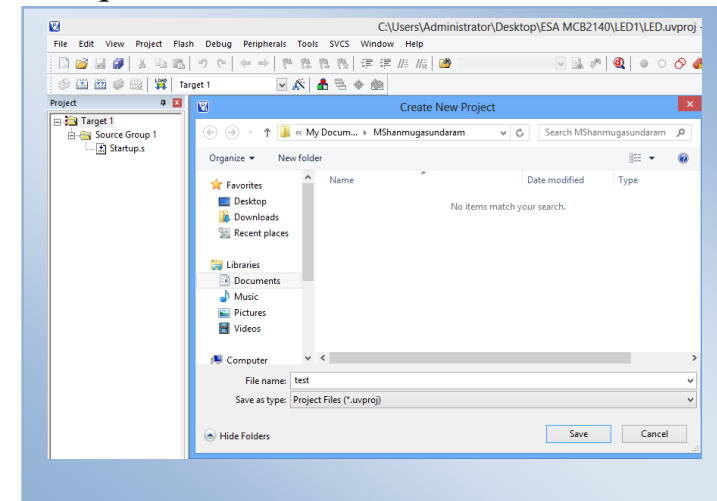
Step 8



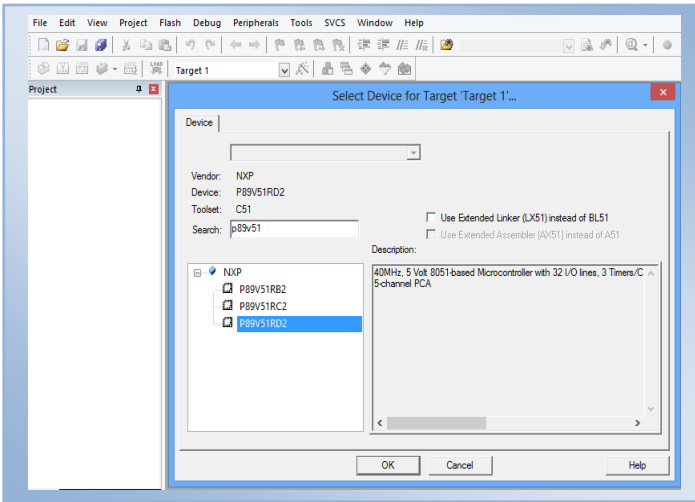
Step 9



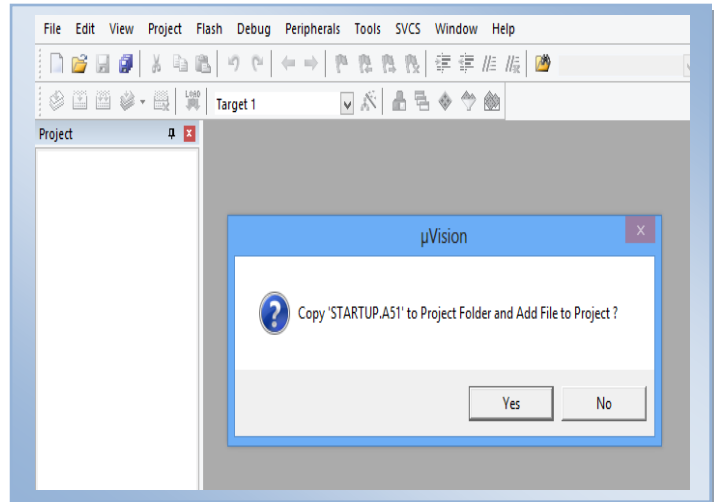
Step 10



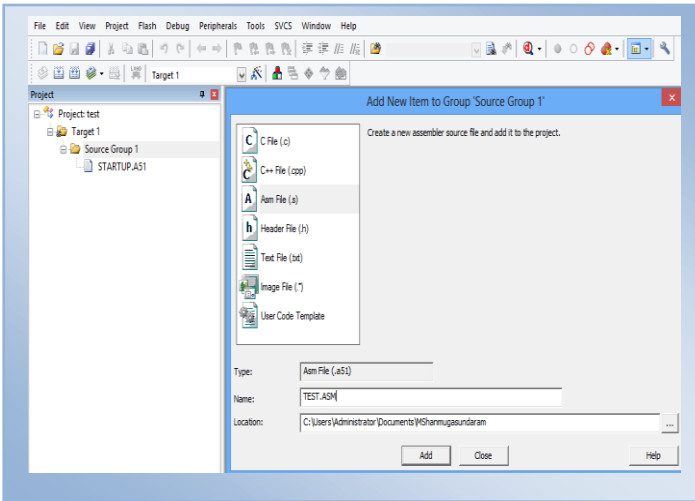
Step 11



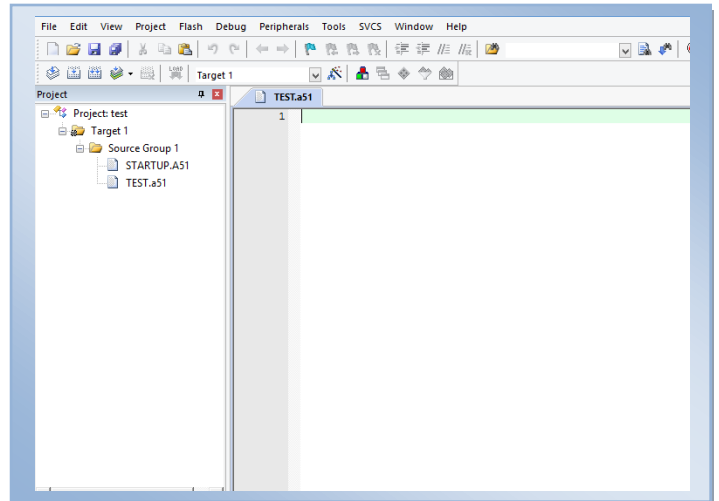
Step 12



Step 13



Step 14



Step No 1 to 7 Represent step by step representation of installing Keil Software.

After Installing Your Desktop may have the keil Icon Double Click on the Icon.

Step No 8 is the original Keil window.

Step No 9 go to the Project Tap select New μ Vision Project click on it.

Step No 10 It's asking the project name -Give Meaning full project name

then give "OK". Upper case and lower case not a problem.

Step No 11 If you Ok then it is showing Device History, Select "NXP Founded by Philips" find the Device no "P89V51RD2" Select then "OK"

Step No 12 After giving Ok one Pop up will come " Copy Startup.A51" Folder add file to the project. Give "Yes". because we need startup.A51" Library files to build our project.

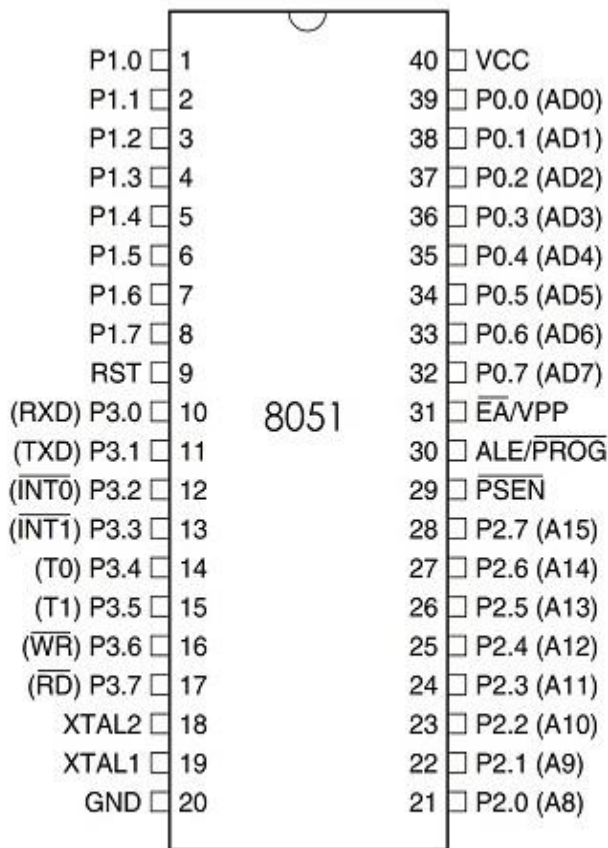
Step No 13 Now your window Left hand side under the project tab, Source group1 tab. Right click on the Source group1 tab add New Item in to group "source group1". Now we can see the project tab under Source Group1 under Startup.A51, under our given file name.

Step No 14 Now Go to File Tab, Click on New and save with file extension is .ASM (eg:TEST.asm)

Step No 15 The window is ready to write and execute.

Chapter 6

8051 hardware and its parts



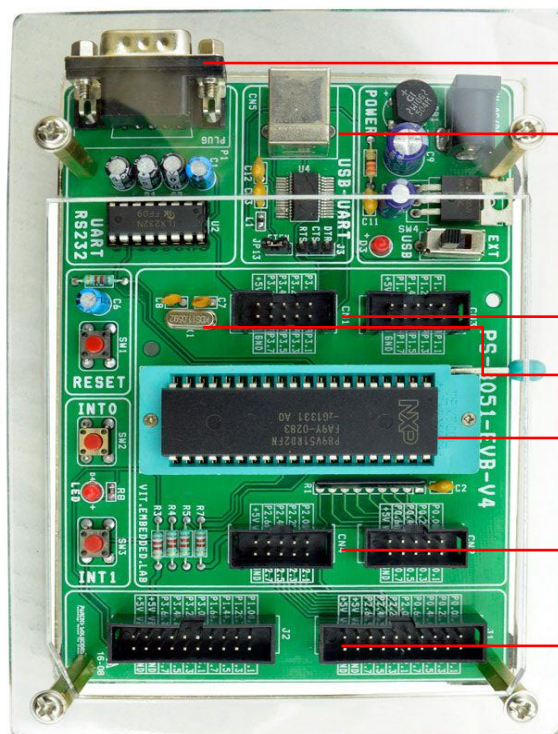
Internal Memory

Microcontroller must have the memory for

Program code – ROM

Variable data – RAM

8051 – Pin Diagram



RS232INTERFACE

USB POWERED
INTERFACE

PORTS (P1,P3)

11.0592MHZ CRYSTAL
OSCILLATOR

P89V51RD2
MICROCONTROLLER CHIP

PORTS (P0,P2)

MOTOR DRIVER PORTS

Exercise – 1

Program 1

Aim:

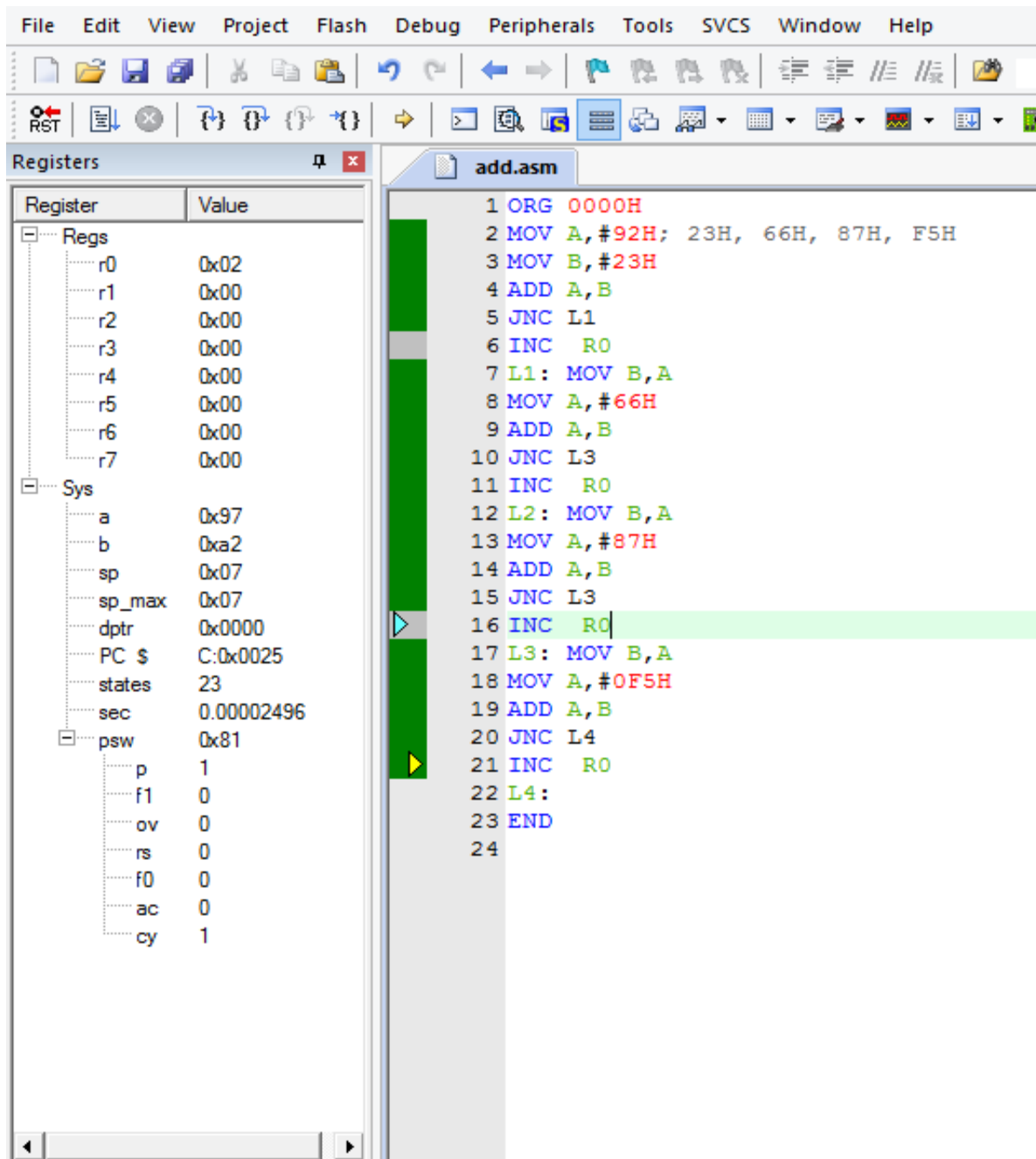
Write and assemble a program to add the following data and then use the simulator to examine the CY flag. 92H, 23H, 66H, 87H, F5H

Assembly Program:

```
ORG 0000H
MOV A,#92H;
MOV B,#23H
ADD A,B
JNC L1
INC R0
L1: MOV B,A
MOV A,#66H
ADD A,B
JNC L2
INC R0
L2: MOV B,A
MOV A,#87H
ADD A,B
JNC L3
INC R0
L3: MOV B,A
MOV A,#0F5H
ADD A,B
JNC L4
INC R0
L4:
END
```

Output :

By adding 92H, 23H, 66H, 87H, F5H will get 297 is the answer
Register A holding 97 and carry flag is 1.



The screenshot shows an assembly editor interface. On the left, the 'Registers' window displays the state of various registers. On the right, the 'add.asm' file contains assembly code. The assembly code performs a series of additions: 92H + 23H, then 66H, then 87H, and finally F5H, with jumps and increments in between. The final state of Register A is 97 (0x97) and the carry flag (cy) is 1.

Register	Value
Regs	
r0	0x02
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x97
b	0xa2
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x0025
states	23
sec	0.00002496
psw	0x81
p	1
f1	0
ov	0
rs	0
f0	0
ac	0
cy	1

```
1 ORG 0000H
2 MOV A,#92H; 23H, 66H, 87H, F5H
3 MOV B,#23H
4 ADD A,B
5 JNC L1
6 INC R0
7 L1: MOV B,A
8 MOV A,#66H
9 ADD A,B
10 JNC L3
11 INC R0
12 L2: MOV B,A
13 MOV A,#87H
14 ADD A,B
15 JNC L3
16 INC R0
17 L3: MOV B,A
18 MOV A,#0F5H
19 ADD A,B
20 JNC L4
21 INC R0
22 L4:
23 END
24
```

Program 2

Aim:

Write and assemble a program to load values into each of registers R0 - R4 and then push each of these registers onto the stack. Single-step the program, and examine the stack and the SP register after the execution of each instruction.

Assembly Program:

```
ORG 000H
MOV R0,#01H
MOV R1,#07H
MOV R2,#09H
MOV R3,#05H
MOV R4,#03H
PUSH 0
PUSH 1
PUSH 2
PUSH 3
PUSH 4
END
```


Output :

The screenshot displays an assembly editor interface with three main panels: Registers, Code, and Memory.

Registers Panel: A table showing the state of various registers.

Register	Value
Regs	
r0	0x01
r1	0x07
r2	0x09
r3	0x05
r4	0x03
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x0b
sp_max	0x0b
dptr	0x0000
PC \$	C:0x0012
states	13
sec	0.00001411
psw	0x00

Code Panel (pushpop.asm): Displays assembly instructions.

```
1 ORG 000H
2 MOV R0,#01H
3 MOV R1,#07H
4 MOV R2,#09H
5 MOV R3,#05H
6 MOV R4,#03H
7 PUSH 0
8 PUSH 1
9 PUSH 2
10 PUSH 3
11 PUSH 4
12 H:SJMP H
13 END
14
```

Memory Panel (Memory 2): Shows a memory dump starting at address d:00H.

Address	Value
D:0x00:	01 07 09 05 03 00 00 00
D:0x08:	01 07 09 05 00 00 00 00
D:0x10:	00 00 00 00 00 00 00 00
D:0x18:	00 00 00 00 00 00 00 00
D:0x20:	00 00 00 00 00 00 00 00
D:0x28:	00 00 00 00 00 00 00 00
D:0x30:	00 00 00 00 00 00 00 00
D:0x38:	00 00 00 00 00 00 00 00
D:0x40:	00 00 00 00 00 00 00 00
D:0x48:	00 00 00 00 00 00 00 00
D:0x50:	00 00 00 00 00 00 00 00
D:0x58:	00 00 00 00 00 00 00 00
D:0x60:	00 00 00 00 00 00 00 00
D:0x68:	00 00 00 00 00 00 00 00
D:0x70:	00 00 00 00 00 00 00 00
D:0x78:	00 00 00 00 00 00 00 00

Program 3

Aim:

Write and assemble a program to

- (a) Set SP = 0D,
- (b) Put a different value in each of RAM locations 0D, 0C, 0B, 0A, 09, and 08,
- (c) POP each stack location into registers R0 - R4.

Use the simulator to single-step and examine the registers, the stack, and the stack pointer.

Assembly Program:

```
ORG 0000H
MOV SP,#0DH
MOV 0DH, #10H
MOV 0CH, #11H
MOV 0BH, #12H
MOV 0AH, #13H
MOV 09H,#14H
MOV 08,#16H
POP 0
POP 1
POP 2
POP 3
POP 4
END
```

Output :

The screenshot displays the Keil uVision IDE interface. The main window shows the assembly file **STACK.ASM** with the following code:

```
1 ORG 0000H
2 MOV SP, #0DH
3 MOV 0DH, #10H
4 MOV 0CH, #11H
5 MOV 0BH, #12H
6 MOV 0AH, #13H
7 MOV 09H, #14H
8 MOV 08, #16H
9 POP 0
10 POP 1
11 POP 2
12 POP 3
13 POP 4
14 END
15
```

On the left, the **Registers** window shows the state of various registers:

Register	Value
Regs	
r0	0x10
r1	0x11
r2	0x12
r3	0x13
r4	0x14
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x08
sp_max	0x0d
dptr	0x0000
PC \$	C:0x001F
states	24
sec	0.00002604
psw	0x00
p	0
f1	0
ov	0
rs	0
f0	0
ac	0
cy	0

At the bottom right, the **Memory 2** window shows the memory dump starting at address **d:00H**:

Address	Value
D:0x00:	10 11 12 13 14 00 00 00
D:0x08:	16 14 13 12 11 10 00 00
D:0x10:	00 00 00 00 00 00 00 00
D:0x18:	00 00 00 00 00 00 00 00
D:0x20:	00 00 00 00 00 00 00 00
D:0x28:	00 00 00 00 00 00 00 00
D:0x30:	00 00 00 00 00 00 00 00
D:0x38:	00 00 00 00 00 00 00 00
D:0x40:	00 00 00 00 00 00 00 00
D:0x48:	00 00 00 00 00 00 00 00
D:0x50:	00 00 00 00 00 00 00 00

Program 4

Aim:

Write and assemble a program to load values into each of registers R0 - R4 and then push each of these registers onto the stack and pop them back. Single-step the program, and examine the stack and the SP register after the execution of each instruction.

Assembly Program:

```
ORG 0000H
MOV R0,#25H
MOV R1,#35H
MOV R2,#45H
MOV R3,#55H
MOV R4,#65H
PUSH 0
PUSH 1
PUSH 2
PUSH 3
PUSH 4
POP 0
POP 1
POP 2
POP 3
POP 4
END
```

Output :

The screenshot displays a microcontroller development IDE with the following components:

- Registers Window:** A table showing the current values of various registers.
- Assembly Code Window (STACK1.ASM):** A list of assembly instructions with line numbers.
- Memory Window (Memory 2):** A table showing the contents of memory locations starting from address 0x0000.

Register	Value
r0	0x65
r1	0x55
r2	0x45
r3	0x35
r4	0x25
r5	0x00
r6	0x00
r7	0x00
a	0x00
b	0x00
sp	0x07
sp_max	0x0c
dptr	0x0000
PC	0x001E
states	25
sec	0.00002713
psw	0x00

```
1 ORG 0000H
2 MOV R0, #25H
3 MOV R1, #35H
4 MOV R2, #45H
5 MOV R3, #55H
6 MOV R4, #65H
7 PUSH 0
8 PUSH 1
9 PUSH 2
10 PUSH 3
11 PUSH 4
12 POP 0
13 POP 1
14 POP 2
15 POP 3
16 POP 4
17 END
18
```

Address	Value
D:0x00	65 55 45 35 25 00 00 00
D:0x08	25 35 45 55 65 00 00 00
D:0x10	00 00 00 00 00 00 00 00
D:0x18	00 00 00 00 00 00 00 00
D:0x20	00 00 00 00 00 00 00 00
D:0x28	00 00 00 00 00 00 00 00
D:0x30	00 00 00 00 00 00 00 00
D:0x38	00 00 00 00 00 00 00 00
D:0x40	00 00 00 00 00 00 00 00
D:0x48	00 00 00 00 00 00 00 00
D:0x50	00 00 00 00 00 00 00 00
D:0x58	00 00 00 00 00 00 00 00

Exercise – 2

Data Transfer

Program 1

Aim:

Write a program to transfer a string of data from code space starting at address 200H to RAM locations inside the CPU starting at 40H.

The data is as shown below:

MYDATA: DB "VIT University" Using the simulator, single-step through the program and examine the data transfer and registers

Assembly Program:

```
ORG 000H
MOV A,#00H
MOV DPTR,#200H
MOV R1,#0EH
MOV R0,#40H
LOOP:CLR A
MOVC A,@A+DPTR
MOV @R0,A
INC DPTR
INC R0
DJNZ R1,LOOP
HERE:SJMP HERE
ORG 200H
DB "VIT UNIVERSITY"
END
```

Output :

The screenshot displays the Keil uVision IDE interface with the following components:

- Registers Window:** Shows the status of registers r4 through r7, and system registers (Sys) a, b, sp, dptr, P..., st..., sec, and psw. Most registers contain 0x00, while sp is 0x07 and dptr is 0x0000.
- Disassembly Window:** Shows the assembly code for 'vituniversity.asm'. The code is as follows:

```
1 ORG 000H
2 MOV A, #00H
3 MOV DPTR, #200H
4 MOV R1, #0EH
5 MOV R0, #40H
6 LOOP: CLR A
7 MOVC A, @A+DPTR
8 MOV @R0, A
9 INC DPTR
10 INC R0
11 DJNZ R1, LOOP
12 HERE: SJMP HERE
13 ORG 200H
14
15 DB "VIT UNIVERSITY"
16 END
17
```
- Command Window:** Shows the command 'Running with Code Size Limit: 2K' and the load path 'Load "C:\\Users\\Administrator\\Documents\\MShanmugasundaram\\'.
- Memory Window:** Shows the memory address 0x200 and the contents of memory starting from C:0x0200: VIT UNIVERSITY..

Program 2

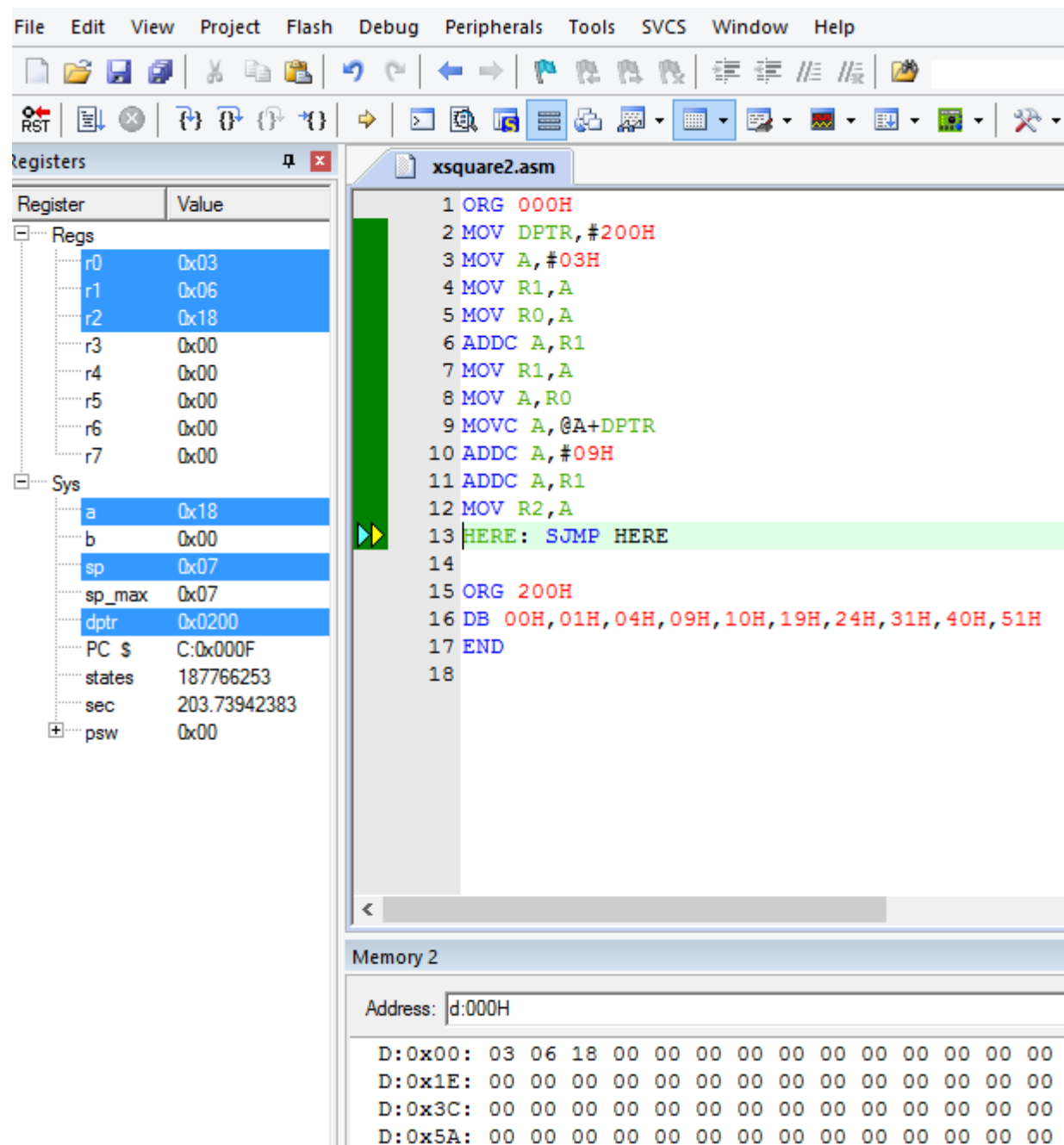
Aim:

Write a program to calculate y where $y = x^2 + 2x + 9$. x is between 0 and 9 and the look-up table for x^2 is located at the address (code space) of 200H. Register R0 has the x , and at the end of the program R2 should have y . Use the simulator to change the x value and single-step through the program, examining the registers as you go.

Assembly Program:

```
ORG 000H
MOV DPTR,#200H
MOV A,#03H
MOV R1,A
MOV R0,A
ADDC A,R1
MOV R1,A
MOV A,R0
MOVC A,@A+DPTR
ADDC A,#09H
ADDC A,R1
MOV R2,A
HERE: SJMP HERE
ORG 200H
DB 00H,01H,04H,09H,10H,19H,24H,31H,40H,51H
END
```


Output :



Exercise – 3

Multi-byte Arithmetic Operation

Program 1

Aim:

Write a program to add 10 bytes of data and store the result in registers R2 and R3. The bytes are stored in the ROM space starting at 200H. The data would look as follows:

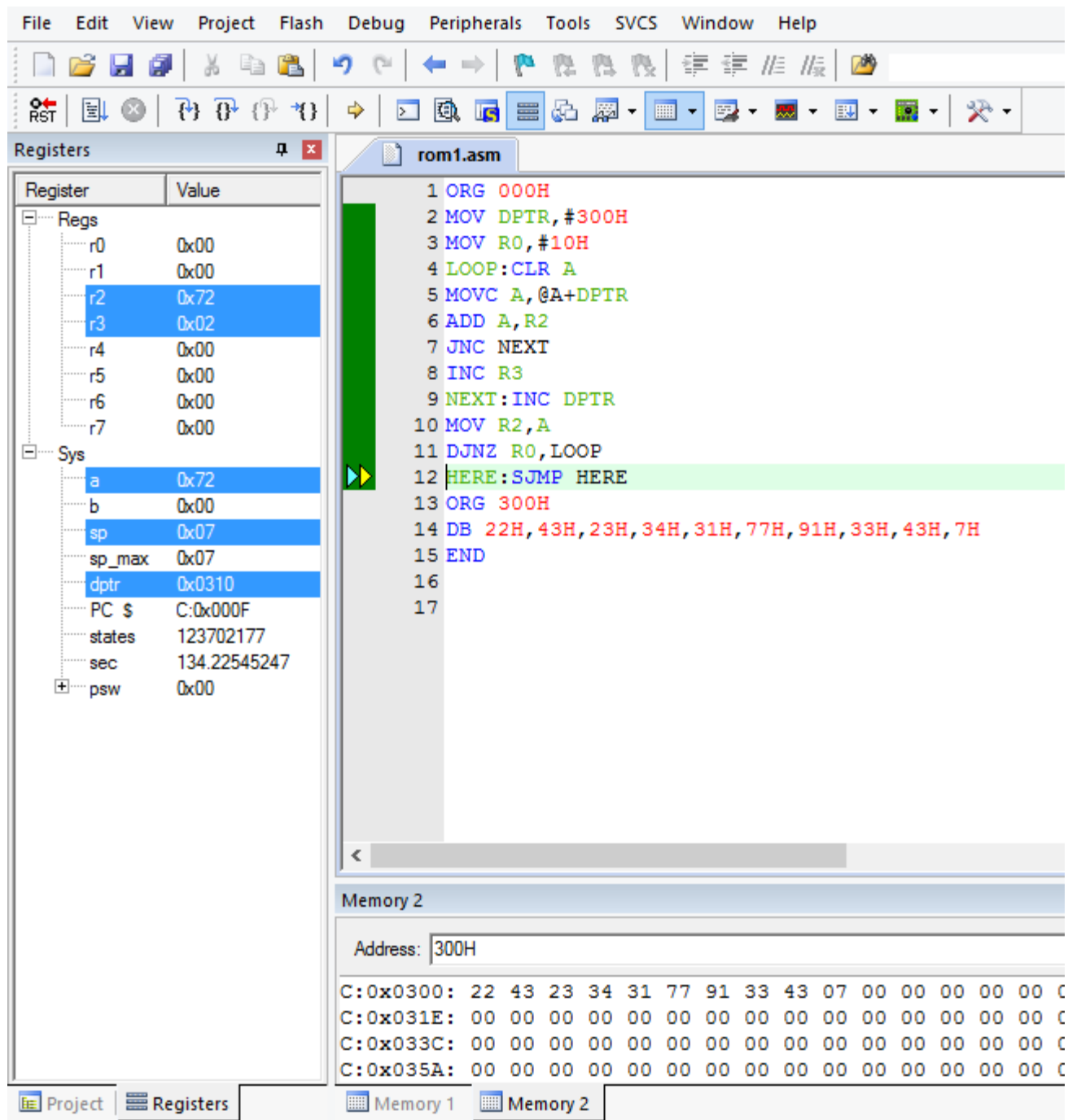
MYDATA: DB; picks your own data.

Notice that you must first bring the data from ROM space into the CPU's RAM, then add them together. Use a simulator to single-step the program and examine the data.

Assembly program:

```
ORG 000H
MOV DPTR,#300H
MOV R0,#10H
LOOP:CLR A
MOVC A,@A+DPTR
ADD A,R2
JNC NEXT
INC R3
NEXT:INC DPTR
MOV R2,A
DJNZ R0,LOOP
HERE:SJMP HERE
ORG 300H
DB 22H,43H,23H,34H,31H,77H,91H,33H,43H,7H
END
```

Output :



Program 2

Aim:

Write a program to add 10 bytes of BCD data and store the result in R2 and R3. The bytes are stored in ROM space starting at 300H. The data would look as follows:

MYDATA: DB; picks your own data.

Notice that you must first bring the data from ROM space into the CPU's RAM, then add them together. Use a simulator to single-step the program and examine the data.

Assembly program:

```
ORG 000H
MOV DPTR,#300H
MOV R0,#10H
LOOP:CLR A
MOVC A,@A+DPTR
ADD A,R2
DA A
JNC NEXT
INC R3
NEXT:INC DPTR
MOV R2,A
DJNZ R0,LOOP
HERE:SJMP HERE
ORG 300H
DB 22H,43H,23H,34H,31H,77H,91H,33H,43H,7H
END
```

Output :

The screenshot displays an 8051 assembly IDE with the following components:

- Registers Window:** Shows the state of 8051 registers. General registers r0-r7 are at 0x00 or 0x04. Special Function Registers (a, b, sp, sp_max, dptr, PC, states, sec, psw) are also shown with their current values.
- Assembly Code Window (rom1.asm):** Contains the following assembly code:

```
1 ORG 000H
2 MOV DPTR, #300H
3 MOV R0, #10H
4 LOOP: CLR A
5 MOVC A, @A+DPTR
6 ADD A, R2
7 DA A
8 JNC NEXT
9 INC R3
10 NEXT: INC DPTR
11 MOV R2, A
12 DJNZ R0, LOOP
13 HERE: SJMP HERE
14 ORG 300H
15 DB 22H, 43H, 23H, 34H, 31H, 77H, 91H, 33H, 43H, 7H
16 END
```
- Memory Window (Memory 2):** Shows a memory dump starting at address 300H. The data at 0x0300 matches the DB instruction in the code.

Register	Value
r0	0x00
r1	0x00
r2	0x04
r3	0x04
r4	0x00
r5	0x00
r6	0x00
r7	0x00
a	0x04
b	0x00
sp	0x07
sp_max	0x07
dptr	0x0310
PC	0x0010
states	125923367
sec	136.63559787
psw	0x01

Address	Value
C:0x0300	22 43 23 34 31 77 91 33 43 07 00 00 00 00
C:0x031E	00 00 00 00 00 00 00 00 00 00 00 00 00 00
C:0x033C	00 00 00 00 00 00 00 00 00 00 00 00 00 00
C:0x035A	00 00 00 00 00 00 00 00 00 00 00 00 00 00

Exercise – 4

Data Conversion

Program 1

Aim

Write a program to get a byte of hex data from P1, convert it to decimal, and then to ASCII. For example, if P1 has FBH, which is equal to 251 in decimal, after conversion we will have 32H, 35H, and 31H. Place the ASCII result in RAM locations starting at 40H. Using a simulator, single-step the program and examine the data.

Assembly program

```
ORG 0000H
MOV P1,#0FBH
MOV R0,#40H
MOV A,P1
LOOP:MOV B,#10
DIV AB
XCH A,B
ADD A,#30H
MOV @R0,A
XCH A,B
INC R0
JNZ LOOP
END
```

Output :

The screenshot displays an assembly IDE with the following components:

- Registers Window:** A table showing the state of various registers.
- Code Editor:** Displays assembly code for 'dataconver.asm' with a green execution cursor at line 12.
- Memory Window:** Shows memory data at address d:40h.

Register	Value
Regs	
r0	0x43
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x32
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x0015
states	43
sec	0.00004666
psw	0x00

```
1  ORG 0000H
2  MOV P1, #0FBH
3  MOV R0, #40H
4  MOV A, P1
5  LOOP: MOV B, #10
6  DIV AB
7  XCH A, B
8  ADD A, #30H
9  MOV @R0, A
10 XCH A, B
11 INC R0
12 JNZ LOOP
13 END
14
15
```

Memory 1

Address: d:40h

D:0x40: 152.....

D:0xB5:

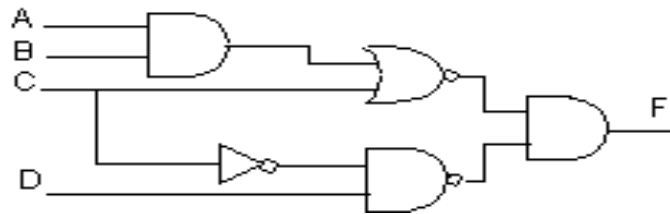
Exercise – 5

Digital Circuit

Program 1

Aim

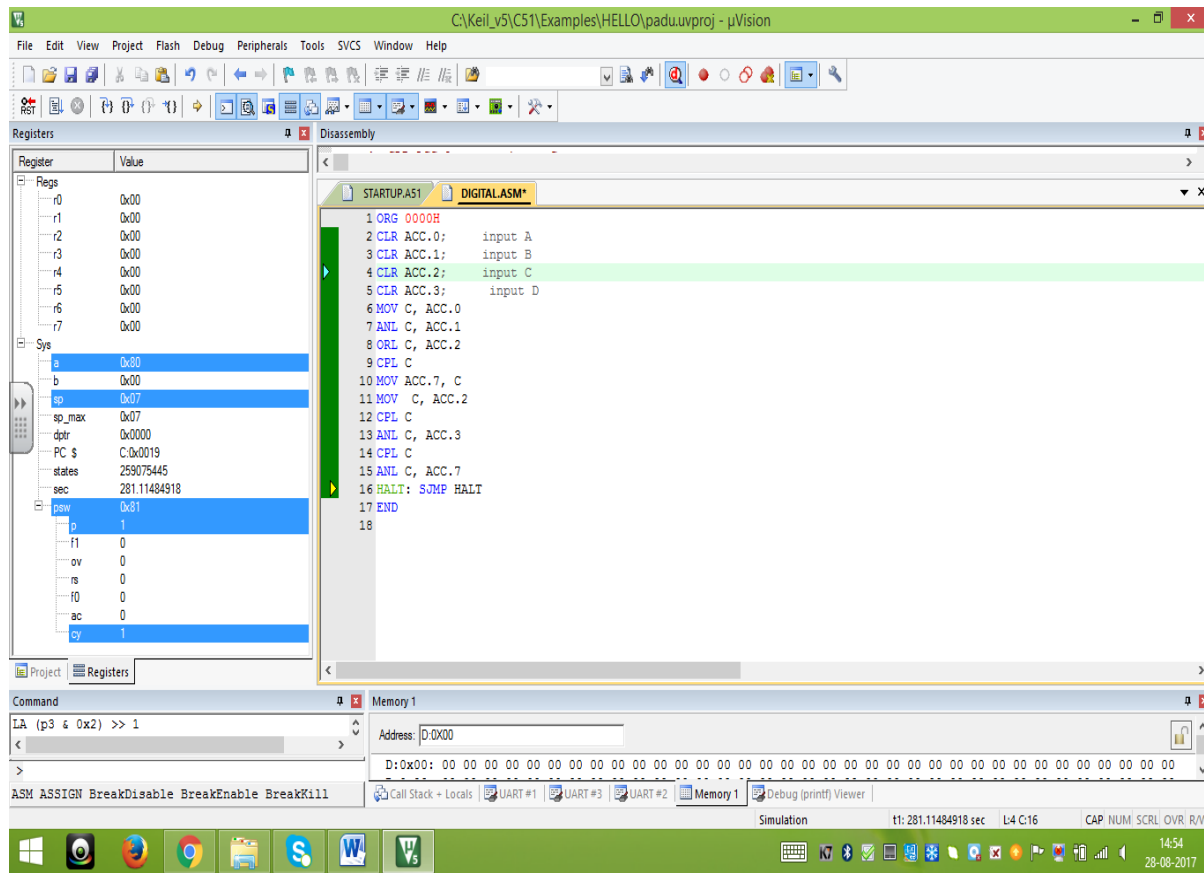
To develop an assembly code for 8051 Microcontroller, to implement the given digital circuit using KEIL development tool.



Assembly program

```
ORG 0000H
SETB ACC.0;   input A
SETB ACC.1;   input B
CLR ACC.2;    input C
SETB ACC.3;   input D
MOV C, ACC.0
ANL C, ACC.1
ORL C, ACC.2
CPL C
MOV ACC.7, C
MOV C, ACC.2
CPL C
ANL C, ACC.3
CPL C
ANL C, ACC.7; Final output (F)
HALT: SJMP HALT
END
```


Keil Simulation



RESULT:

S.NO	A	B	C	D	Output (F)
1	0	0	0	0	
2	0	0	0	1	
3	0	0	1	0	
4	0	0	1	1	
5	0	1	0	0	
6	0	1	0	1	
7	0	1	1	0	
8	0	1	1	1	
9	1	0	0	0	
10	1	0	0	1	
11	1	0	1	0	
12	1	0	1	1	
13	1	1	0	0	
14	1	1	0	1	
15	1	1	1	0	
16	1	1	1	1	

Exercise – 6

Port Programming

Program 1

Aim

Write and assemble a program to toggle all the bits of P0, P1, and P2 continuously by sending 55H and AAH to these ports. Put a time delay between the "on" and "off" states. Then using the simulator, single-step through the program and examine the ports. Do not single-step through the time delay call.

Assembly Program:

```
ORG 0000H
HERE : MOV P0,#55H
MOV P1,#55H
MOV P2,#55H
ACALL DELAY
MOV P0,#0AAH
MOV P1,#0AAH
MOV P2,#0AAH
ACALL DELAY
SJMP HERE
DELAY : MOV R1,#04H
BACK: MOV R2,#20H
AGAIN: DJNZ R2,AGAIN
DJNZ R1,BACK
RET
END
```

Output :

The screenshot shows an IDE with the following components:

- Registers:** A table showing the state of registers. The 'Sys' section includes 'PC \$' at address C:0x001C with value 657495197.
- Disassembly:** A list of assembly instructions for 'port.asm'. Line 14, 'AGAIN: DJNZ R2, AGAIN', is highlighted in green.
- Parallel Port 0:** A configuration window showing 'Port 0' with value 0xAA and 'Pins' set to 0x00.
- Parallel Port 1:** A configuration window showing 'Port 1' with value 0xAA and 'Pins' set to 0xAA.

Register	Value
r0	0x00
r1	0x04
r2	0x12
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x09
sp_...	0x09
dptr	0x0000
PC \$	C:0x001C
states	657495197
sec	197.24857882
psw	0x00

```
1
2  ORG 0000H
3  HERE : MOV P0, #55H
4  MOV P1, #55H
5  MOV P2, #55H
6  ACALL DELAY
7  MOV P0, #0AAH
8  MOV P1, #0AAH
9  MOV P2, #0AAH
10 ACALL DELAY
11 SJMP HERE
12 DELAY : MOV R1, #04H
13 BACK:  MOV R2, #20H
14 AGAIN: DJNZ R2, AGAIN
15 DJNZ R1, BACK
16 RET
17 END
18
```

Parallel Port 0

Port 0: 0xAA 7 Bits 0

Pins: 0x00

Parallel Port 1

Port 1: 0xAA 7 Bits 0

Pins: 0xAA

The screenshot shows the same IDE with updated values:

- Disassembly:** Line 14, 'AGAIN: DJNZ R2, AGAIN', remains highlighted in green.
- Parallel Port 0:** The 'Port 0' value has been updated to 0x55.
- Parallel Port 1:** The 'Port 1' value has been updated to 0x55.

Register	Value
r0	0x00
r1	0x04
r2	0x12
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x09
sp_...	0x09
dptr	0x0000
PC \$	C:0x001C
states	657495197
sec	197.24857882
psw	0x00

```
1
2  ORG 0000H
3  HERE : MOV P0, #55H
4  MOV P1, #55H
5  MOV P2, #55H
6  ACALL DELAY
7  MOV P0, #0AAH
8  MOV P1, #0AAH
9  MOV P2, #0AAH
10 ACALL DELAY
11 SJMP HERE
12 DELAY : MOV R1, #04H
13 BACK:  MOV R2, #20H
14 AGAIN: DJNZ R2, AGAIN
15 DJNZ R1, BACK
16 RET
17 END
18
```

Parallel Port 0

Port 0: 0x55 7 Bits 0

Pins: 0x00

Parallel Port 1

Port 1: 0x55 7 Bits 0

Pins: 0x55

Program 2

Aim

Test the 8051's ports for input operation as follows. Connect each bit of P1 (P1.0 - P1.7) of the 8051 to DIP switches. Read Appendix C.2 very carefully to make sure that you do not damage the port. Then write and run a program to get data from P1 and send it to P2. Any change of status of the switches connected to P1 will be instantly reflected on LEDs connected to P2. The testing program could look like this.

Assembly Program:

```
Org 0000h
MOV A,#0FFH
MOV P1,A
HERE :MOV A,P1
MOV P2,A
SJMP HERE
END
```

Output :

The screenshot displays a microcontroller development IDE with the following components:

- Menu Bar:** File, Edit, View, Project, Flash, Debug, Peripherals, Tools, SVCS, Window, Help.
- Toolbar:** Includes icons for file operations, navigation, and execution.
- Registers Window:** Shows the state of registers and system variables.

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0xfa
b	0x00
sp	0x07
sp_max	0x07
dptr	0x0000
PC \$	C:0x0008
states	16
sec	0.00001736
psw	0x00
- Assembly Editor (PORT1.ASM):**

```
1 Org 0000h
2 MOV A, #0FFH
3 MOV P1, A
4 HERE :MOV A, P1
5 MOV P2, A
6 SJMP HERE
7 END
8
```
- Parallel Port 1 Configuration:**
 - Port 1: 7 Bits, 0
 - P1: 0xFA (bits 0-6 checked)
 - Pins: 0xFA (bits 0-6 checked)
- Parallel Port 2 Configuration:**
 - Port 2: 7 Bits, 0
 - P2: 0xFA (bits 0-6 checked)
 - Pins: 0xFA (bits 0-6 checked)

Exercise – 7

Timer Programming

Program 1

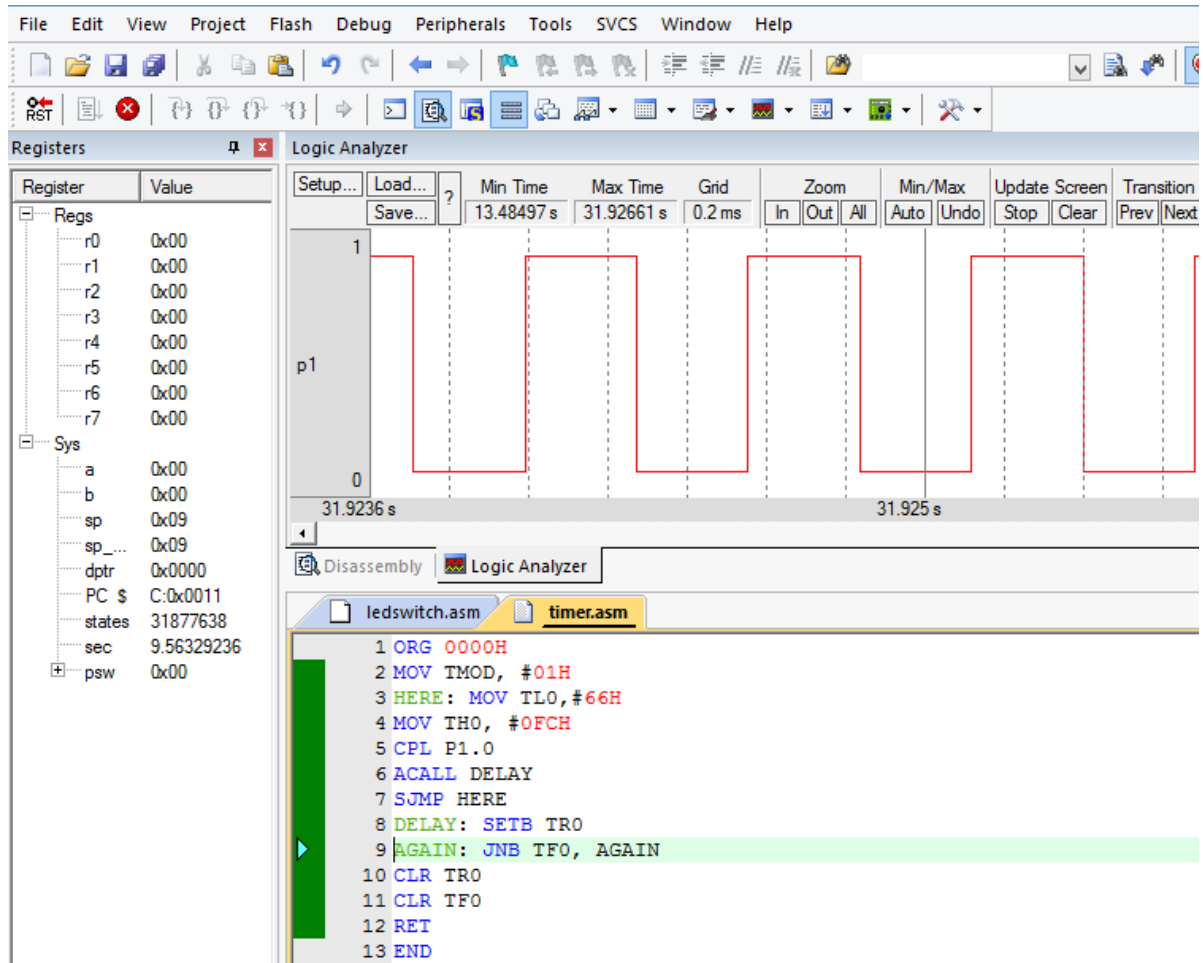
Aim

Write a program using timer 0 to generate a 500 Hz square wave frequency on one of the pins of P1. Then examine the frequency using the KEIL IDE inbuilt Logic Analyzer

Assembly Program

```
ORG 0000H
MOV TMOD, #01H
HERE: MOV TL0, #66H
MOV TH0, #0FCH
CPL P1.0
ACALL DELAY
SJMP HERE
DELAY: SETB TR0
AGAIN: JNB TF0, AGAIN
CLR TR0
CLR TF0
RET
END
```

Output :



Exercise –8

Counter Programming

Program 1

Aim

Assuming that clock pulses are fed into pin T1, write a program for counter 1 in mode 2 to count the pulses and display the state of the TL1 count on P2, which connects to 8 LEDs.

Assembly Program:

```
ORG 0000H
MOV TMOD,#01100000B
MOV TH1,#0
SETB P3.5
AGAIN: SETB TR1
BACK: MOV A,TL1
MOV P2,A
JNB TF1,Back
CLR TR1
CLR TF1
SJMP AGAIN
END
```


Output :

The screenshot displays a microcontroller development environment with the following components:

- Registers Window:** A table showing the state of various registers.
- Assembly Code Window (COUNT123.ASM):** Displays the assembly program being executed.
- Parallel Port 2 Configuration:** A dialog box showing settings for Port 2.
- Parallel Port 3 Configuration:** A dialog box showing settings for Port 3.

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x04
b	0x00
sp	0x07
sp_max	0x07
dptr	0x0000
PC	0x000E
states	19809732
sec	21.49493490
psw	0x01

```
1  ORG 0000H
2  MOV TMOD, #01100000B
3  MOV TH1, #0
4  SETB P3.5
5  AGAIN: SETB TR1
6  BACK: MOV A, TL1
7  MOV P2, A
8  JNB TF1, Back
9  CLR TR1
10 CLR TF1
11 SJMP AGAIN
12 END
13
```

Parallel Port 2 Configuration:

- Port 2: 7 Bits, 0
- P2: 0x04
- Pins: 0x04

Parallel Port 3 Configuration:

- Port 3: 7 Bits, 0
- P3: 0xFF
- Pins: 0xFF

Program 2

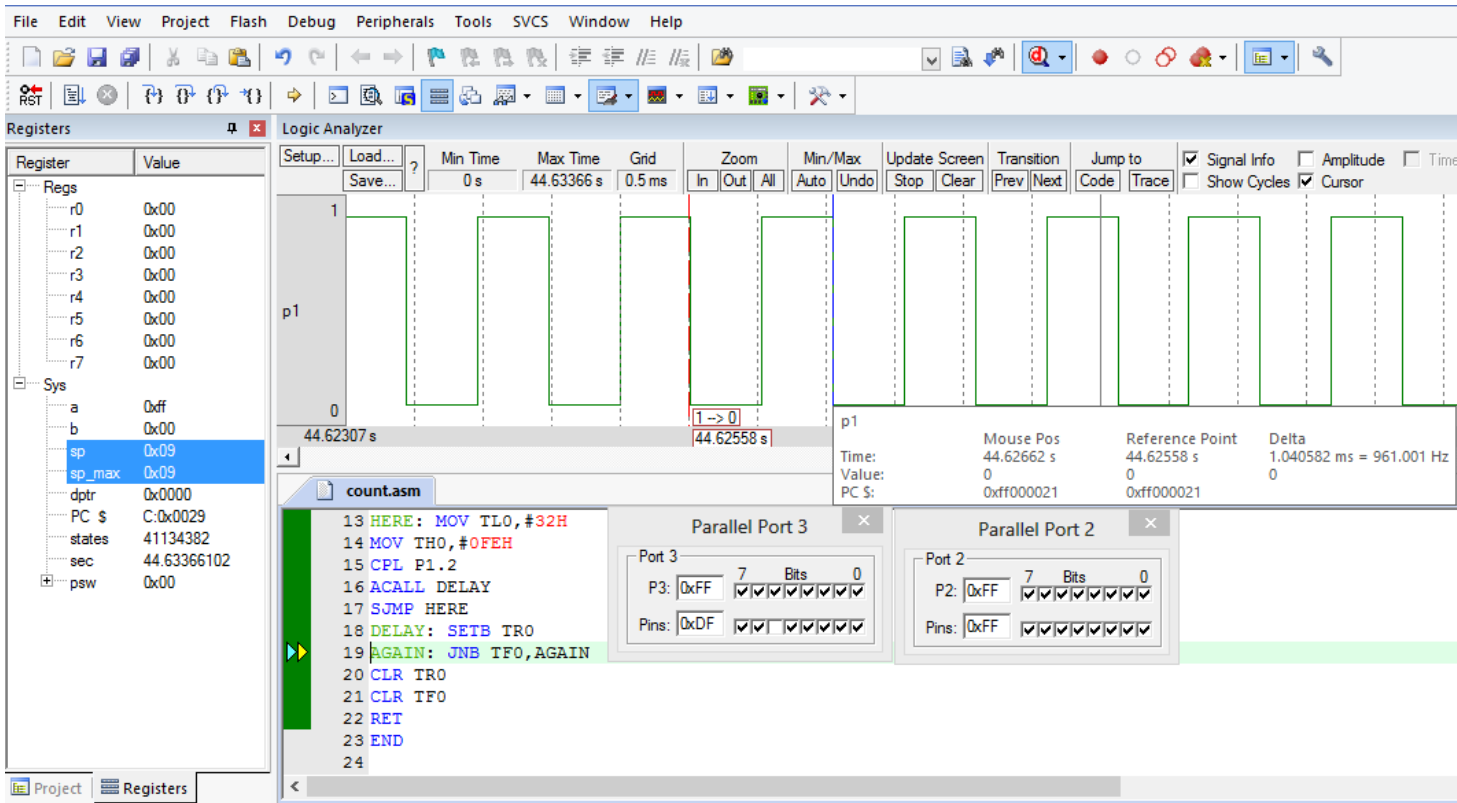
Aim:

Use Counter 1 in mode 2 and after 10 number of counts on TL1, generate a SQUARE waveform of 1 KHz on P1.2 by using Timer 0 in mode 1, show the counts in TL1 on port 2.

Assembly Program:

```
ORG 0000H
MOV TMOD,#60H
MOV TH1,#0f5H
MOV TL1,#0F5H
SETB P3.5
SETB TR1
BACK:MOV A,TL1
MOV P2,A
JNB TF1,BACK
CLR TR1
CLR TF1
MOV TMOD,#01H
HERE: MOV TL0,#32H
MOV TH0,#0FEH
CPL P1.2
ACALL DELAY
SJMP HERE
DELAY: SETB TR0
AGAIN: JNB TF0,AGAIN
CLR TR0
CLR TF0
RET
END
```

Output :



Exercise –9

Serial Communication

Program 1

Aim:

Write a program to transfer a string of data from code space starting at address 200H to RAM locations inside the CPU starting at 40H.

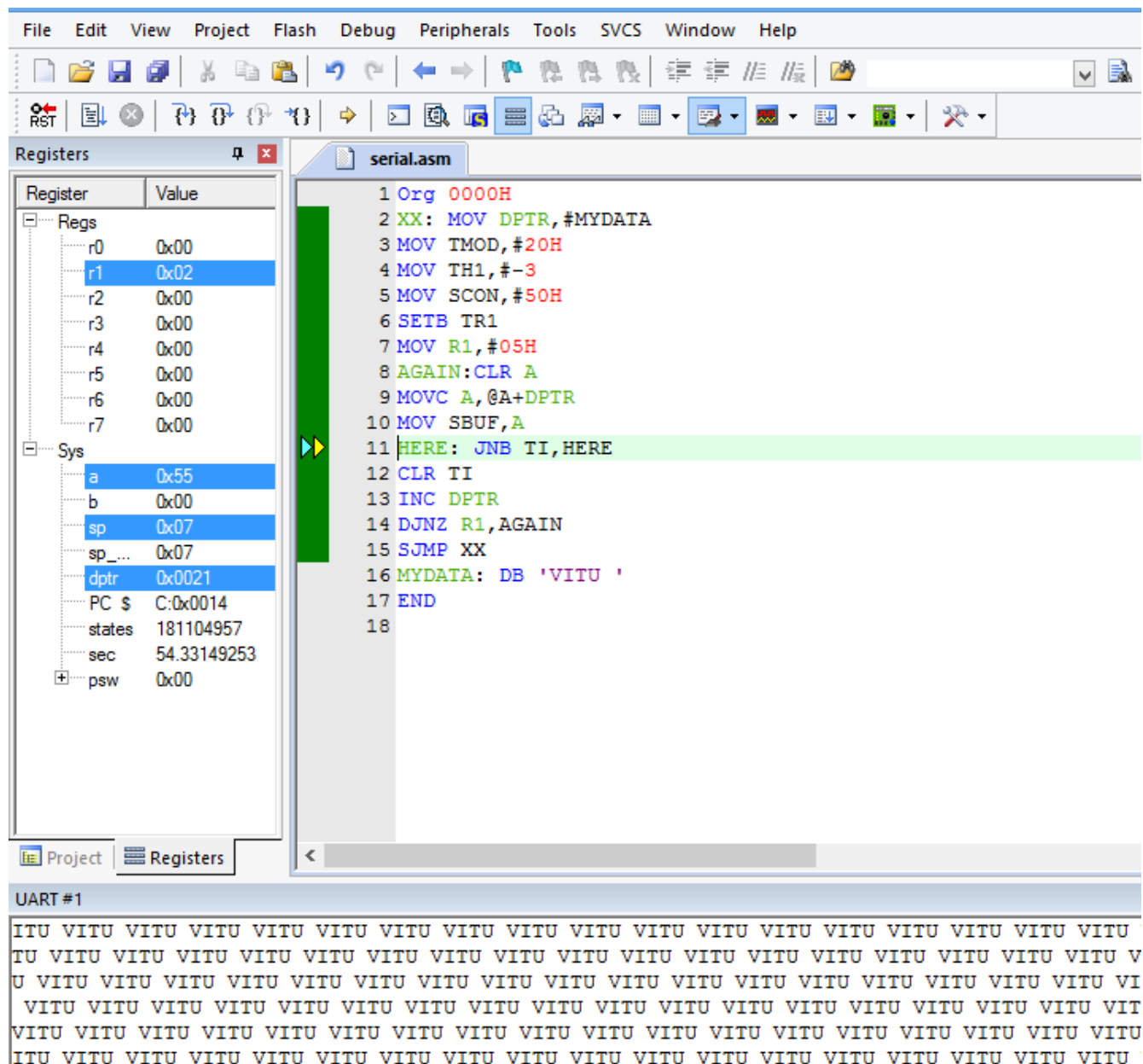
The data is as shown below:

MYDATA: DB "VITU" Using the simulator, single-step through the program and examine the data transfer and registers

Assembly program:

```
Org 0000H
XX: MOV DPTR,#MYDATA
MOV TMOD,#20H
MOV TH1,#-3
MOV SCON,#50H
SETB TR1
MOV R1,#05H
AGAIN:CLR A
MOVC A,@A+DPTR
MOV SBUF,A
HERE: JNB TI,HERE
CLR TI
INC DPTR
DJNZ R1,AGAIN
SJMP XX
MYDATA: DB 'VITU '
END
```

Output :



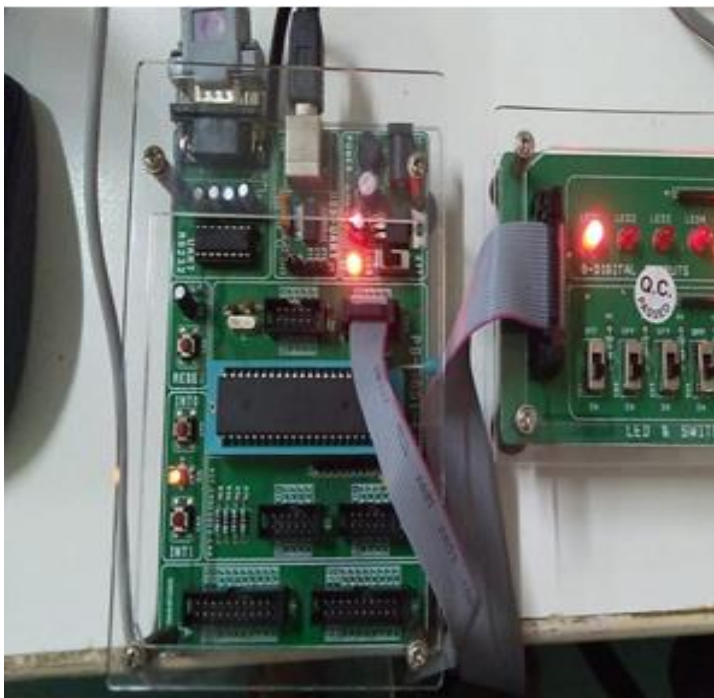
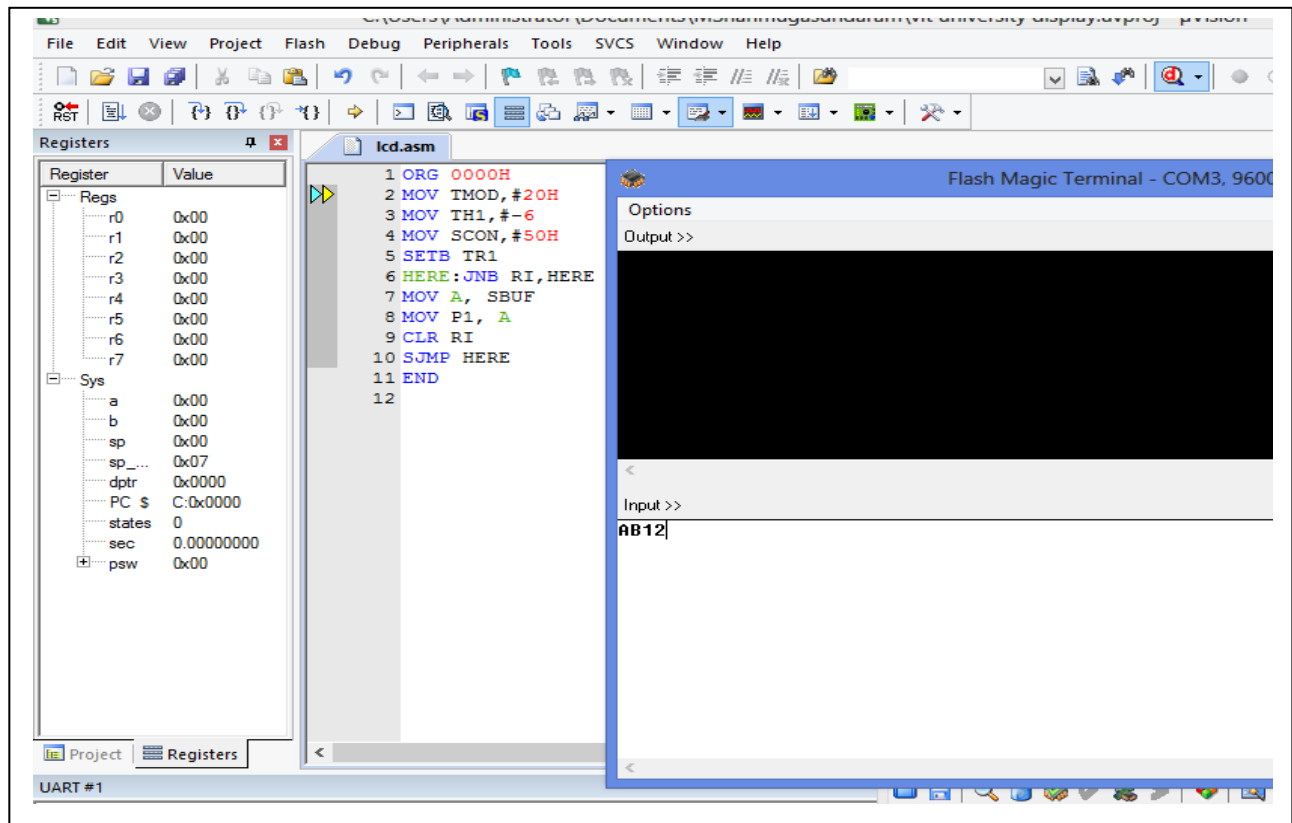
Program 2

Aim:

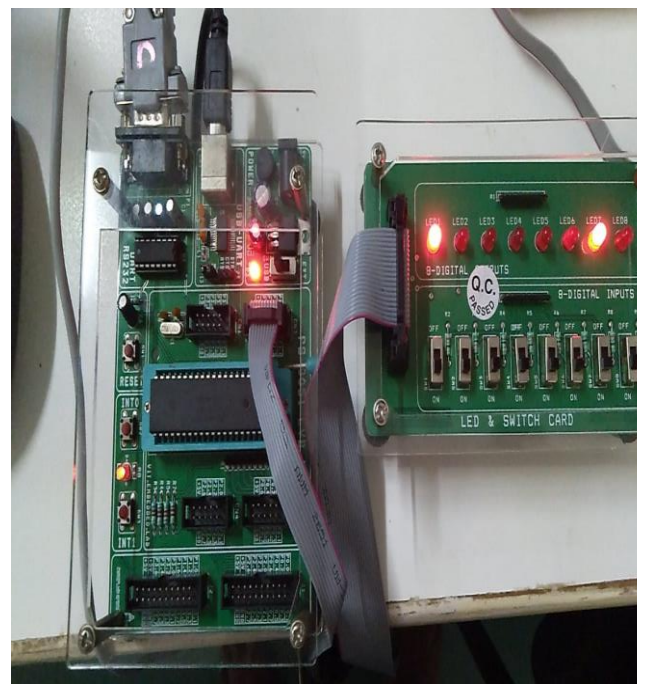
After making sure that program 1 works properly, write a program to get data from the PC and display it on an LCD. If you do not have an LCD connected to the 8051, connect P1 to LEDs and observe the incoming characters. As you press a key on the PC's keyboard, the character is sent to the 8051 serially and is displayed on LEDs. The characters displayed on LEDs are in ASCII (binary).

Assembly program:

```
ORG 0000H
MOV TMOD,#20H
MOV TH1,#-6
MOV SCON,#50H
SETB TR1
HERE:JNB RI,HERE
MOV A, SBUF
MOV P1, A
CLR RI
SJMP HERE
END
```



Character "1" Display in Binary



Character "A" Display in Binary

Exercise –10

Interrupt Programming

Program 1

Aim:

Write an 8051 program to get data from one port and send it to another port continuously while an interrupt will do the following: One of the timers will toggle the PX.Y bit every 100 microseconds.

Assembly Program:

```
ORG 0000H
LJMP MAIN
ORG 000BH
CPL P2.1
RETI
ORG 0030H
MAIN:MOV TMOD,#02H
MOV P0,#0FFH
MOV TH0,#-92
MOV IE,#10000010B
SETB TR0
BACK:MOV A,P0
MOV P1,A
SJMP BACK
END
```


Output :

The screenshot displays a microcontroller development environment with the following components:

- Registers Window:** Shows the state of various registers. The 'a' register (Accumulator) is highlighted with a value of 0xff. Other registers like r0-r7, sp, dptr, PC, and psw are also visible.
- Logic Analyzer:** Displays a timing diagram for signal 'p2'. The signal is a square wave alternating between 0 and 1. The time scale ranges from 0s to 30.06722s.
- Assembly Code Window (interrupt.asm):** Shows the following code:

```
1 ORG 0000H
2 LJMP MAIN
3 ORG 000BH
4 CPL P2.1
5 RETI
6 ORG 0030H
7 MAIN:MOV TMOD,#02H
8 MOV P0,#0FFH
9 MOV TH0,#-92
10 MOV IE,#10000010B
11 SETB TR0
12 BACK:MOV A,P0
13 MOV P1,A
14 SJMP BACK
15 END
```
- Parallel Port Configuration Windows:** Three windows are open for Parallel Port 0, Parallel Port 1, and Parallel Port 2. Each window shows the port value (P0: 0xBF, P1: 0xBF, P2: 0xFD) and the pin configuration (Pins: 0xBF, 0xBF, 0xFD).

Program 2

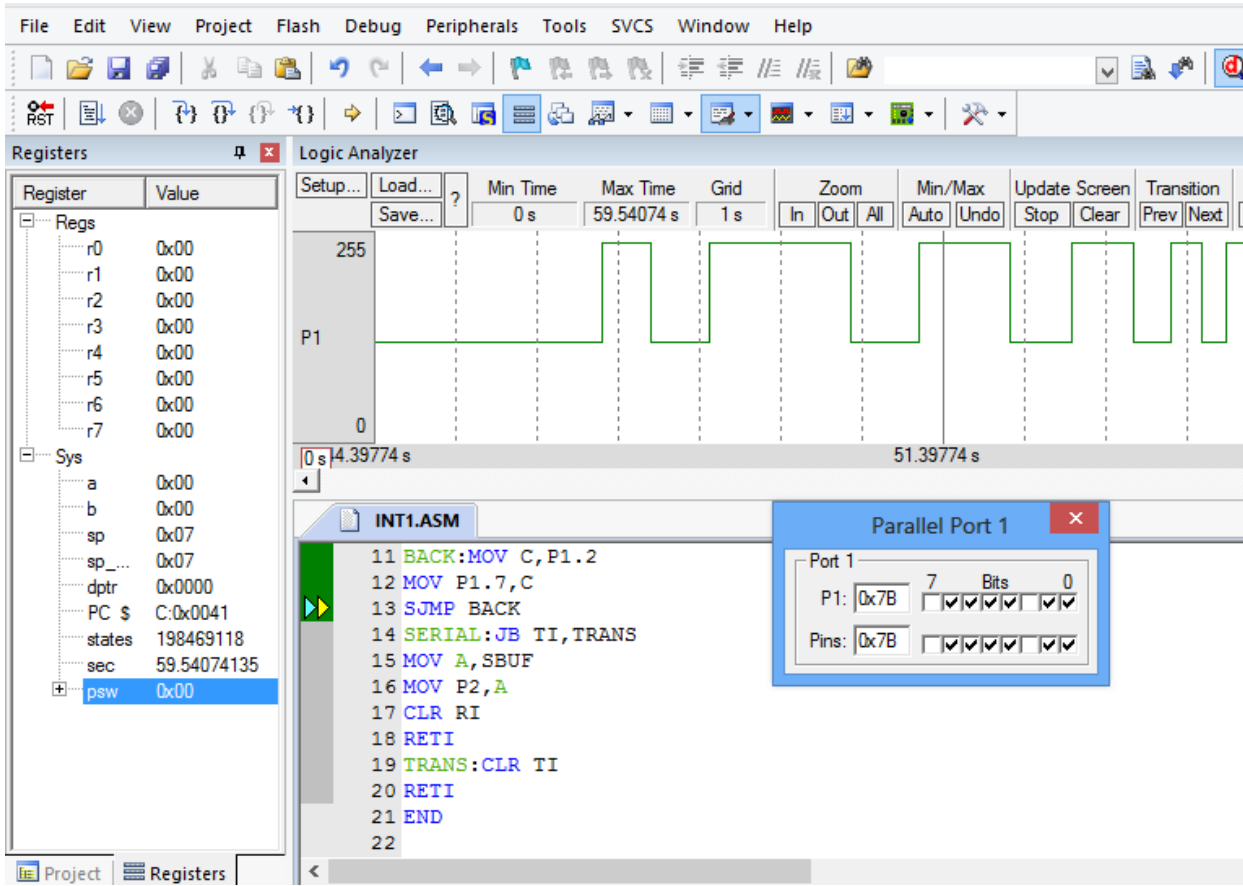
Aim:

Write an 8051 program to get data from a single bit of P1.2 and send it to P1.7 continuously while an interrupt will do the following: A serial interrupt service routine will receive data from a PC and display it on one of the ports. (Either connect LEDs to this port to examine the data coming in or display the received data on an LCD).

Assembly Program:

```
ORG 0000H
LJMP MAIN
ORG 0023H
LJMP SERIAL
ORG 0030H
MAIN:SETB P1.2
MOV TMOD,#20H
MOV TH1,#-3
MOV IE,#10010000B
SETB TR1
BACK:MOV C,P1.2
MOV P1.7,C
SJMP BACK
SERIAL:JB TI,TRANS
MOV A,SBUF
MOV P2,A
CLR RI
RETI
TRANS:CLR TI
RETI
END
```

Output :



Exercise –11

LCD Programming

Program 1

Aim:

Connect the LCD to your 8051 Kit. Then write and run a program to display your name on line 1 of the LCD (first name followed by last name with a space in between).

Note: If you are not monitoring the busy flag of the LCD, put a few milliseconds delay in your program. See the discussion in Chapter 12.

Assembly program:

;P1.0-P1.7 are connected to LCD data pins D0-D7

;P2.0 is connected to RS pin of LCD

;P2.1 is connected to R/W pin of LCD

;P2.2 is connected to E pin of LCD

```
ORG 0000H
```

```
MOV A,#38H
```

```
ACALL CMNWRT
```

```
ACALL DELAY
```

```
MOV A,#0EH
```

```
ACALL CMNWRT
```

```
ACALL DELAY
```

```
MOV A,#01H
```

```
ACALL CMNWRT
```

```
ACALL DELAY
```

```
MOV A,#06H
```

```
ACALL CMNWRT
```

```
ACALL DELAY
```

```
MOV A,#80H
```

```
ACALL CMNWRT
```

```
ACALL DELAY
```

```
MOV A,#'V'
```

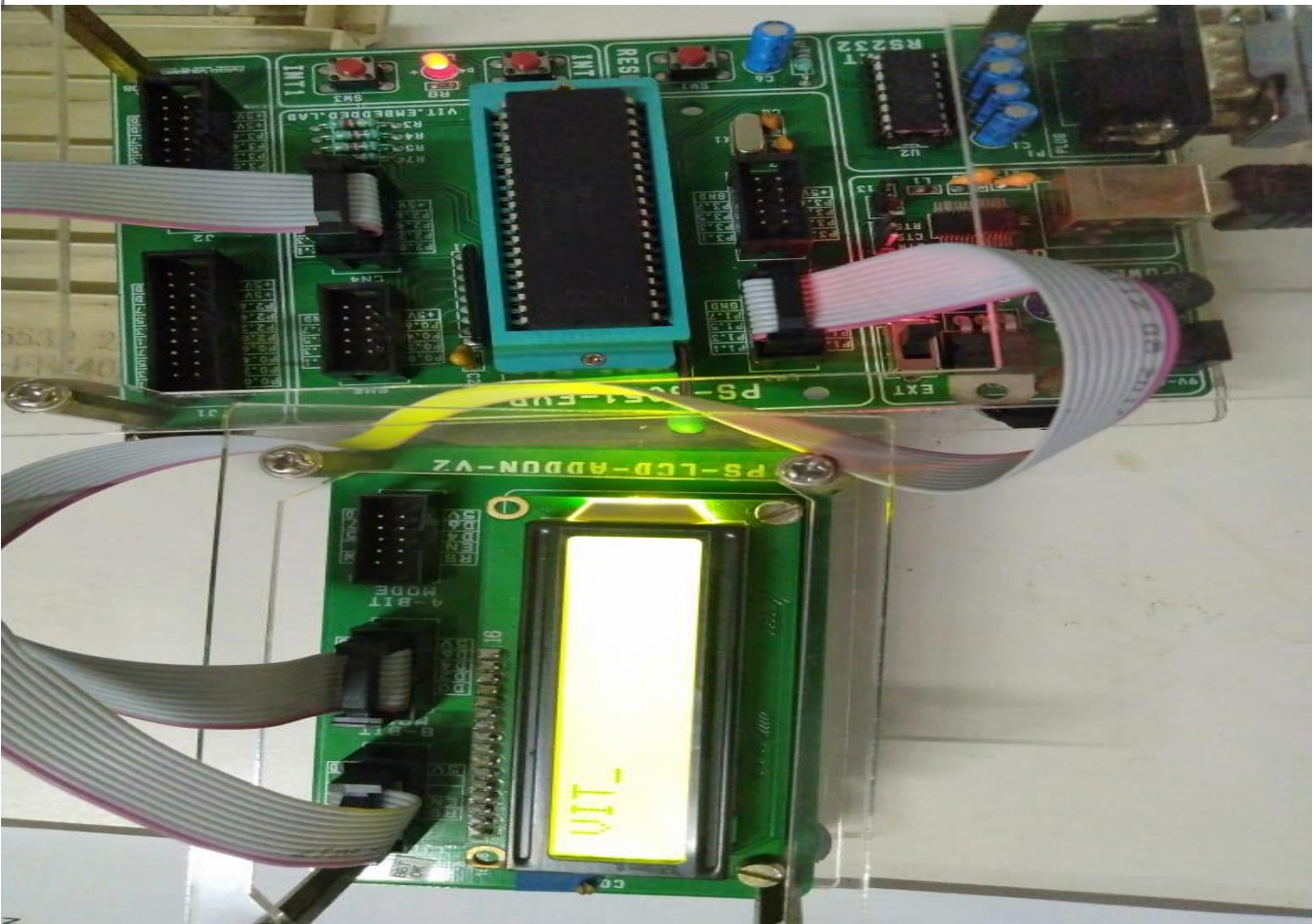
```
ACALL DATWRT
ACALL DELAY
MOV A,#'I'
ACALL DATWRT
ACALL DELAY
MOV A,#'T'
ACALL DATWRT
ACALL DELAY
AGAIN: SJMP AGAIN
CMNWRT:
MOV P1,A
CLR P2.0
CLR P2.1
SETB P2.2
CLR P2.2
RET
DATWRT:
MOV P1,A
SETB P2.0
CLR P2.1
SETB P2.2
CLR P2.2
RET
DELAY: MOV R3,#0FFH
MOV R4,#0FFH
MOV R5,#55H
HERE: DJNZ R5,HERE
HERE1: DJNZ R4, HERE1
HERE2: DJNZ R3,HERE2
RET
END
```

Output :

```
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Project: lcd12
Target 1
Source Group 1
STARTUP.A51
kkk.asm

1 ;P1.0-P1.7 are connected to LCD data pins D0-D7
2 ;P2.0 is connected to RS pin of LCD
3 ;P2.1 is connected to R/W pin of LCD
4 ;P2.2 is connected to E pin of LCD
5
6 ORG 0000H
7 MOV A, #38H
8 ACALL CMNWRT
9 ACALL DELAY
10 MOV A, #0EH
11 ACALL CMNWRT
12 ACALL DELAY
13 MOV A, #01H
14 ACALL CMNWRT
15 ACALL DELAY
16 MOV A, #06H
17 ACALL CMNWRT
18 ACALL DELAY
19 MOV A, #80H
20 ACALL CMNWRT
21 ACALL DELAY
22 MOV A, #'V'
23 ACALL DATWRT
24 ACALL DELAY
25 MOV A, #'I'
26 ACALL DATWRT
27 ACALL DELAY
28 MOV A, #'T'
29 ACALL DATWRT

Build Output
creating hex file from "..\\..\\Desktop\\lcd123"...
"..\\..\\Desktop\\lcd123" - 0 Error(s), 3 Warning(s).
Build Time Elapsed: 00:00:00
```



Assembly program:

```
ORG 0H
MOV A,#38H
ACALL CMDWRT
ACALL DELAY
MOV A,#0EH
ACALL CMDWRT
ACALL DELAY
MOV A,#01H
ACALL CMDWRT
ACALL DELAY
MOV A,#06H
ACALL CMDWRT
ACALL DELAY
MOV A,#084H
ACALL CMDWRT
ACALL DELAY
MOV A,#'V'
ACALL DATAWRT
ACALL DELAY
MOV A,#'I'
ACALL DATAWRT
ACALL DELAY
MOV A,#'T'
ACALL DATAWRT
ACALL DELAY
MOV A,#'V'
ACALL DATAWRT
ACALL DELAY
MOV A,#'L'
ACALL DATAWRT
ACALL DELAY
MOV A,#'R'
ACALL DATAWRT
ACALL DELAY
MOV A,#0C4H
ACALL CMDWRT
```

ACALL DELAY

MOV A,#'M'

ACALL DATAWRT

ACALL DELAY

MOV A,#'C'

ACALL DATAWRT

ACALL DELAY

MOV A,#' '

ACALL DATAWRT

ACALL DELAY

MOV A,#'L'

ACALL DATAWRT

ACALL DELAY

MOV A,#'A'

ACALL DATAWRT

ACALL DELAY

MOV A,#'B'

ACALL DATAWRT

ACALL DELAY

AGAIN: SJMP AGAIN

CMDWRT:MOV P1,A

CLR P2.0

CLR P2.1

SETB P2.2

ACALL DELAY

CLR P2.2

RET

DATAWRT:MOV P1,A

SETB P2.0

CLR P2.1

SETB P2.2

ACALL DELAY

CLR P2.2

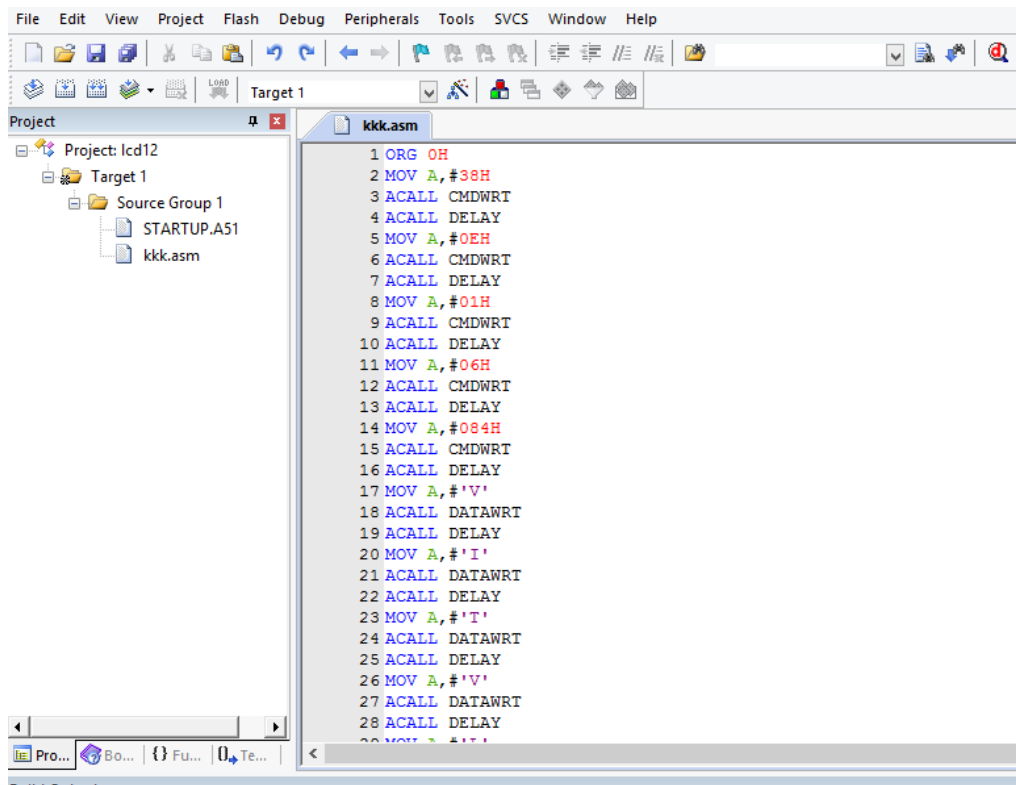
RET

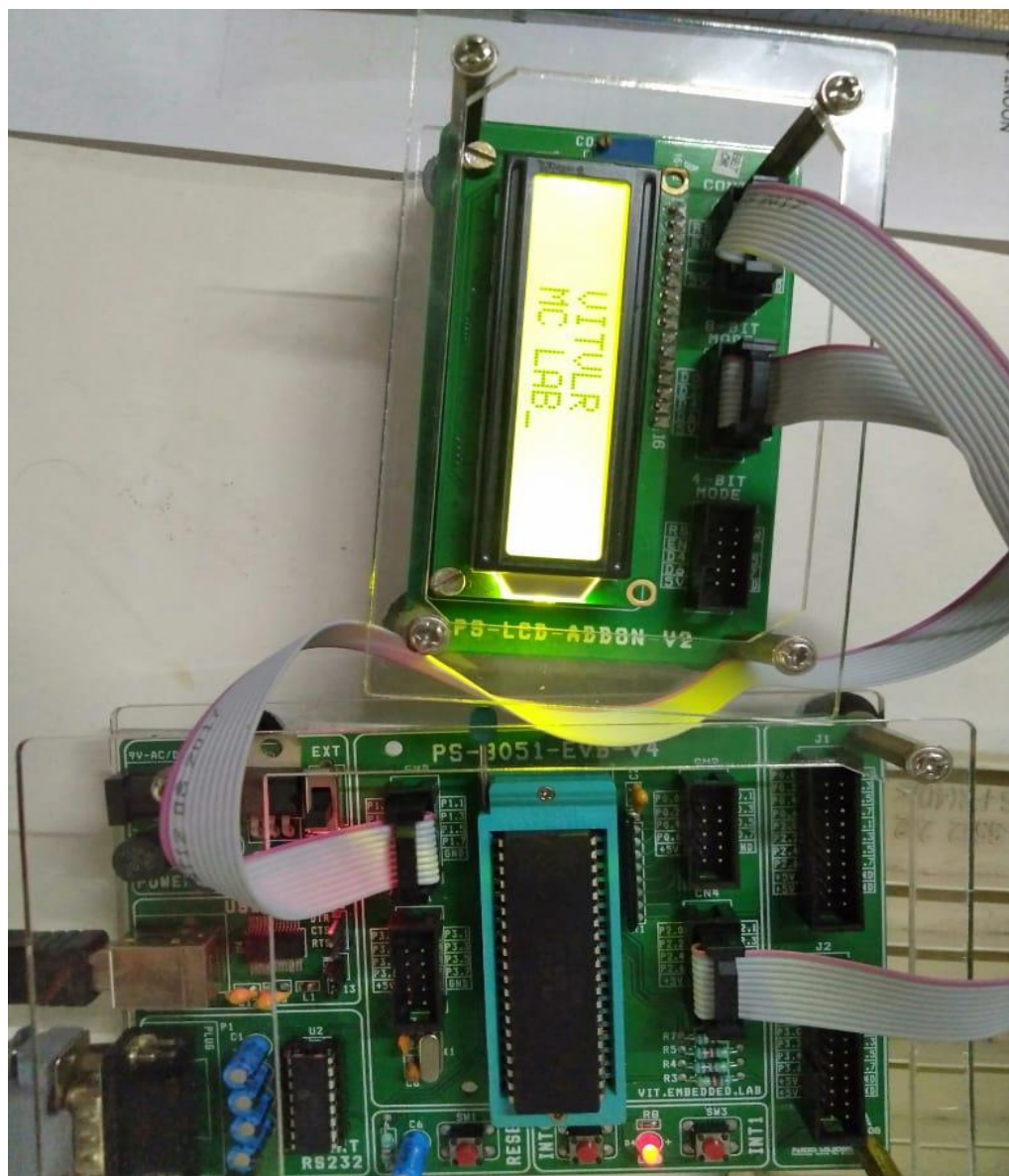
DELAY:MOV R3,#255

HERE2: MOV R4,#255

HERE:DJNZ R4,HERE

DJNZ R3,HERE2
RET
END





Lab Manual Prepared By,



DR.SUMIT KUMAR JINDAL
ASST. PROF (SR) GRADE1
LAB IN-CHARGE
sumit.jindal@vit.ac.in /
VELLORE INSTITUTE OF TECHNOLOGY
VELLORE -632014.



MR.V.KARTHIKEYAN DEEE,BE(ECE).
LAB ASSISTANT -SENSE
karthikeyan.venu@vit.ac.in /
VELLORE INSTITUTE OF TECHNOLOGY
VELLORE -632014.



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Electronics Engineering

Microprocessor & Microcontroller Laboratory

**Microcontroller And Its Application
(ECE3003)
Master Manual**

**Common For
B.Tech (ECE, BMD, & IoT)**



-----XXXX-----

Exercise –12

Keypad Programming

Program 1

Aim:

Connect the LCD and 4 x 4 Keypad to your 8051 Kit. Then write and run a program to get the data from the keypad display it in the LCD.

Assembly program:

```
ORG 0H
START:
ACALL LCDINIT
ACALL GETKEY
AGAIN: SJMP AGAIN
LCDINIT: MOV A,#38H ;INIT. LCD 2 LINES, 5X7 MATRIX
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#0EH ;display on, cursor on
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#01 ;clear LCD
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#06H ;shift cursor right
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#0C0H ;cursor at line 1, pos. 4
ACALL COMNWRT ;call command subroutine
ACALL DELAY ;give LCD some time
MOV A,#'K' ;display letter K
ACALL DATAWRT ;call display subroutine
```

```

ACALL DELAY                ;give LCD some time
MOV  A,#'E'                ;display letter E
ACALL DATAWRT             ;call display subroutine
ACALL DELAY                ;give LCD some time
MOV  A,#'Y'                ;display letter Y
ACALL DATAWRT             ;call display subroutine
ACALL DELAY                ;give LCD some time
MOV  A,#':'                ;display letter :
ACALL DATAWRT             ;call display subroutine
ACALL DELAY                ;give LCD some time
RET
COMNWRT:MOV  P1,A
CLR  P2.0
CLR  P2.1
SETB P2.2
ACALL DELAY
CLR  P2.2
RET
DATAWRT:MOV  P1,A
SETB P2.0
CLR  P2.1
SETB P2.2
ACALL DELAY
CLR  P2.2
RET
DELAY:MOV  R3,#50           ;50 or higher for fast CPUs
HERE2:MOV  R4,#255          ;R4 = 255
HERE:DJNZ  R4,HERE          ;stay until R4 becomes 0
DJNZ  R3,HERE2
RET

;Keyboard subroutine. This program sends the ASCII
;Code for pressed key to P0.1
;P0.4-P0.7 connected to rows, P0.0-P0.3 to column

GETKEY: MOV  P0,#0FH
REP:MOV  P0,#0FH
MOV  A,P0
ANL  A,#0FH
CJNE A,#0FH,OVER
SJMP REP
OVER:ACALL DELAY
MOV  P0,#0FH
MOV  A,P0
ANL  A,#0FH
CJNE A,#0FH,OVER1
SJMP REP

```

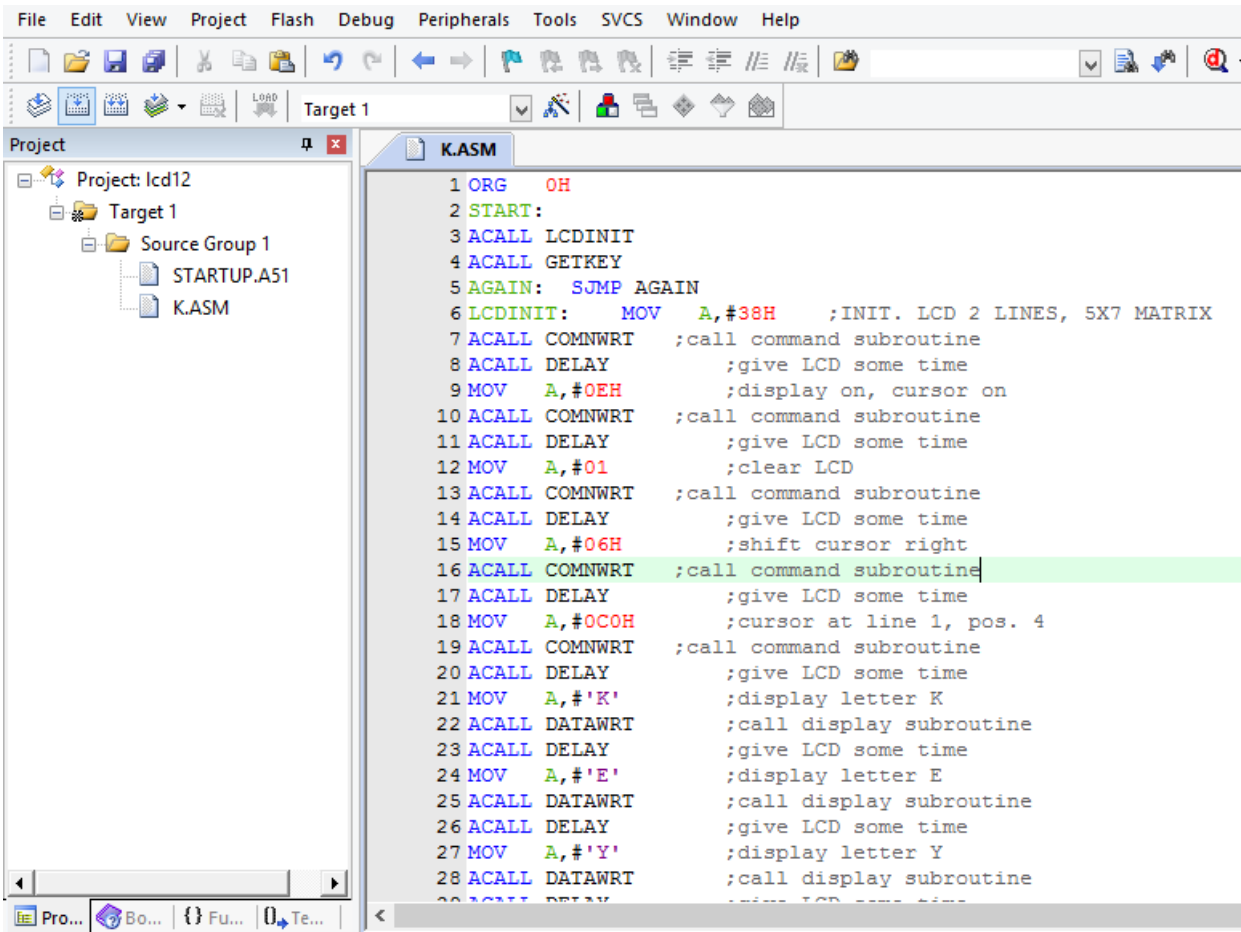
```

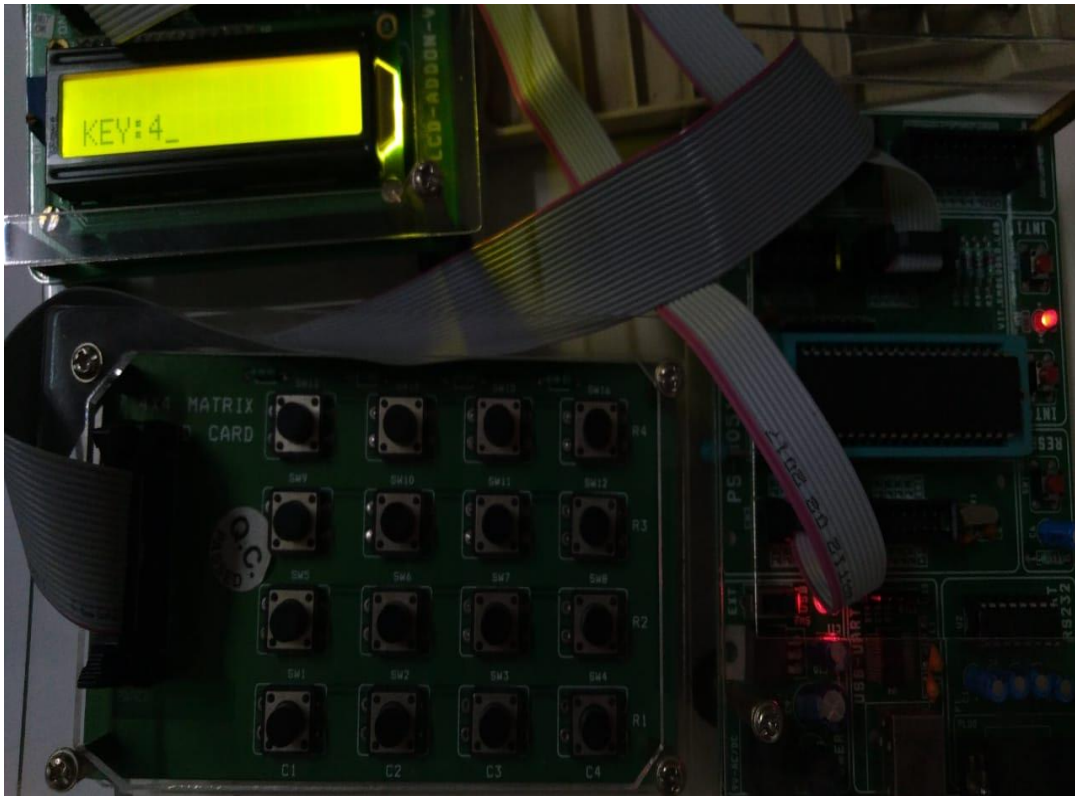
OVER1:CLR P0.4          ;ROW0 SELECTED
SETB P0.5
SETB P0.6
SETB P0.7
MOV A,P0
ANL A,#0FH
CJNE A,#0FH,ROW0
CLR P0.5                ;ROW1 SELECTED
SETB P0.7
SETB P0.6
SETB P0.4
MOV A,P0
ANL A,#0FH
CJNE A,#0FH,ROW1
CLR P0.6                ;ROW2 SELECTED
SETB P0.7
SETB P0.5
SETB P0.4
MOV A,P0
ANL A,#0FH
CJNE A,#0FH,ROW2
CLR P0.7                ;ROW3 SELECTED
SETB P0.4
SETB P0.6
SETB P0.5
MOV A,P0
ANL A,#0FH
CJNE A,#0FH,ROW3
SJMP REP
MOV R0,#04H
ROW0:MOV DPTR,#KCODE0
SJMP FIND
ROW1:MOV DPTR,#KCODE1
SJMP FIND
ROW2:MOV DPTR,#KCODE2
SJMP FIND
ROW3:MOV DPTR,#KCODE3
FIND:RRC A
JNC MATCH
INC DPTR
DJNZ R0,FIND
MATCH:MOV A,#0C4H      ;display pressed key
ACALL COMNVRT          ;
ACALL DELAY
CLR A                  ;set A=0 (match is found)
MOVC A,@A+DPTR         ;get ASCII from table

```



```
ACALL DATAWRT      ;call display subroutine
ACALL DELAY          ;give LCD some time
LJMP REP             ;ASCII LOOK-UP TABLE FOR EACH ROW
ORG 300H
KCODE0: DB 'F','B','8','4' ;ROW 0
KCODE1: DB 'E','A','7','3' ;ROW 1
KCODE2: DB 'D','0','6','2' ;ROW 2
KCODE3: DB 'C','9','5','1' ;ROW 3
END
```





Exercise –12

C Programming

Program 1

Aim:

Write an 8051 C program to toggle all the bits of port P1 continuously with some delay in between. Use Timer 0, 16-bit mode to generate the delay.

C program:

```
#include <reg51.h>
void T0Delay(void);
void main(void)
{
    while (1)
```

```
{  
P1=0x55;  
T0Delay();  
P1=0xAA;  
T0Delay();  
}  
}  
void T0Delay()  
{  
TMOD=0x01;  
TL0=0x00;  
TH0=0x35;  
TR0=1;  
while (TF0==0);  
TR0=0;  
TF0=0;  
}
```

Output :

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Registers

Register	Value
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x09
sp_...	0x09
dptr	0x0000
PC \$	C:0x080B
states	65430960
sec	19.62928996
psw	0x00

toggle.c

```

1 #include <reg51.h>
2 void T0Delay(void);
3 void main(void)
4 {
5     while (1)
6     {
7         P1=0x55;
8         T0Delay();
9         P1=0xAA;
10        T0Delay();
11    }
12 }
13 void T0Delay()
14 {
15     TMOD=0x01;
16     TLO=0x00;
17     TH0=0x35;
18     TR0=1;
19     while (TF0==0);
20     TR0=0;
21     TF0=0;
22 }
23

```

Parallel Port 1

Port 1: 7 Bits 0

P1: 0x55 ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Pins: 0x55 ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help

Registers

Register	Value
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x00
Sys	
a	0x00
b	0x00
sp	0x07
sp_...	0x07
dptr	0x0000
PC \$	C:0x0813
states	389
sec	0.00011670
psw	0x00

toggle.c

```

1 #include <reg51.h>
2 void T0Delay(void);
3 void main(void)
4 {
5     while (1)
6     {
7         P1=0x55;
8         T0Delay();
9         P1=0xAA;
10        T0Delay();
11    }
12 }
13 void T0Delay()
14 {
15     TMOD=0x01;
16     TLO=0x00;
17     TH0=0x35;
18     TR0=1;
19     while (TF0==0);
20     TR0=0;
21     TF0=0;
22 }
23

```

Parallel Port 1

Port 1: 7 Bits 0

P1: 0xAA ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Pins: 0xAA ☒ ☒ ☒ ☒ ☒ ☒ ☒ ☒

Program 2

Aim:

Write an 8051 C program to get a byte of data from P0. If it is less than 100, send it to P1; otherwise, send it to P2.

C program:

```
#include <reg51.h>
void main(void)
{
    unsigned char mybyte;
    P0=0xFF; //make P0 input port
    while (1)
    {
        mybyte=P0; //get a byte from P0
        if (mybyte<100)
            P1=mybyte; //send it to P1
        else
            P2=mybyte; //send it to P2
    }
}
```

Output :

The screenshot displays a microcontroller development environment. The central pane shows a C program named `port.c` with the following code:

```
1 #include <reg51.h>
2 void main(void)
3 {
4     unsigned char mybyte;
5     P0=0xFF; //make P0 input port
6     while (1)
7     {
8         mybyte=P0; //get a byte from P0
9         if (mybyte<100)
10            P1=mybyte; //send it to P1
11        else
12            P2=mybyte; //send it to P2
13    }
14 }
15
```

On the left, the 'Registers' window shows the state of various registers:

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0xfe
Sys	
a	0x00
b	0x00
sp	0x07
sp_...	0x07
dptr	0x0000
PC	0x0815
states	284838709
sec	85.45162125
psw	0x00

On the right, three configuration windows for parallel ports are open:

- Parallel Port 0:** Port 0 is set to 0xFC, and Pins are set to 0xFC. All 8 bits are checked.
- Parallel Port 1:** Port 1 is set to 0xFF, and Pins are set to 0xFF. All 8 bits are checked.
- Parallel Port 2:** Port 2 is set to 0xFC, and Pins are set to 0xFC. All 8 bits are checked.

File Edit View Project Flash Debug Peripherals Tools SVCS Window Help



Registers

Register	Value
Regs	
r0	0x00
r1	0x00
r2	0x00
r3	0x00
r4	0x00
r5	0x00
r6	0x00
r7	0x3f
Sys	
a	0xff
b	0x00
sp	0x07
sp_...	0x07
dptr	0x0000
PC \$	C:0x0811
states	186006560
sec	55.80197358
psw	0xc0

port.c

```
1 #include <reg51.h>
2 void main(void)
3 {
4     unsigned char mybyte;
5     P0=0xFF; //make P0 input port
6     while (1)
7     {
8         mybyte=P0; //get a byte from P0
9         if (mybyte<100)
10            P1=mybyte; //send it to P1
11        else
12            P2=mybyte; //send it to P2
13    }
14 }
15
```

Parallel Port 0

Port 0

P0: 0x1F 7 Bits 0

Pins: 0x1F

Parallel Port 1

Port 1

P1: 0x1F 7 Bits 0

Pins: 0x1F

Parallel Port 2

Port 2

P2: 0x7F 7 Bits 0

Pins: 0x7F